

Vorlesung

Mensch-Maschine-Interaktion

WS 2024/2025



Kapitel D.

Interaktive Bilderzeugung

Priv.-Doz. Dr. Frank Weichert
Lehrstuhl für Computergraphik (Informatik VII)
Technische Universität Dortmund
<https://graphics.cs.tu-dortmund.de>

Hinweis:

„Der Nutzerin/dem Nutzer ist bekannt, dass die nachfolgenden Inhalte und Materialien urheberrechtlich geschützt sind. Die Nutzung ist ausschließlich für den persönlichen Gebrauch im Rahmen von universitären Zwecken zulässig. Insbesondere ist die Vervielfältigung, Bearbeitung, Verbreitung und jede Art der Verwertung sowie die Weitergabe an Dritte nicht gestattet. Zuwiderhandlungen werden zivil- und strafrechtlich verfolgt.“

D. Interaktive Bilderzeugung

Übersicht

D.1. Grundkonzepte

- D.1.1. Einführung
- D.1.2. Bilderzeugungs-Pipelines
- D.1.3. Geometrieprepräsentation
- D.1.4. Geometrieeigenschaften

D.2. Transformation

- D.2.1. Einleitung
- D.2.2. Affine Abbildungen
- D.2.3. Homogene Darstellung

D.3. Beleuchtungsberechnung

- D.3.1. Einleitung
- D.3.2. Beleuchtungsmodell von Phong
- D.3.3. Shading-Modelle

D.4. Clipping und Projektion

- D.4.1. Einleitung
- D.4.2. Clipping
- D.4.3. Picking
- D.4.4. Projektion
- D.4.5. Kamerakalibrierung

D.5. Sichtbarkeit

- D.5.1. Einleitung
- D.5.2. Szenenbasiertes Lösungsverfahren
 - D.5.2.1. Painter's Algorithm
 - D.5.2.2. Binäre Raumzerlegung
- D.5.3. Rasterbildbasierte Lösungsverfahren
 - D.5.3.1. Tiefenpufferverfahren
 - D.5.3.2. Scan-Line-Verfahren
 - D.5.3.3. Bereichsunterteilungsverfahren

D.6. Verrasterung

- D.6.1. Strecken in expliziter Darstellung
- D.6.2. Strecken in impliziter Darstellung
- D.6.3. Anti-Alias-Behandlung
- D.6.4. Dicke Strecken
- D.6.5. Verrasterung von Polygonen
- D.6.6. Beleuchtungsberechnung und Verrasterung
- D.6.7. Textur
- D.6.8. Flächenfüllen

Interaktive Bilderzeugung

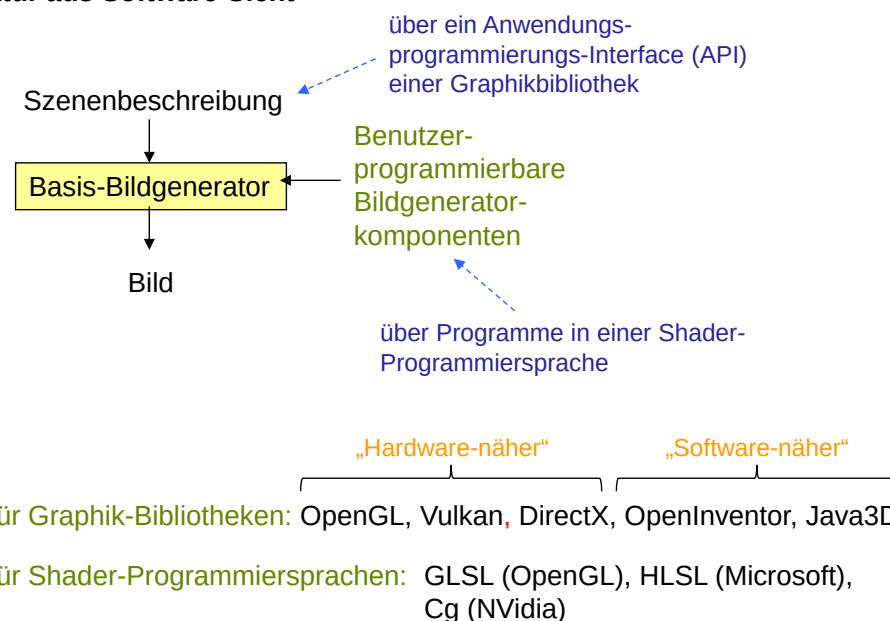
Bilderzeugung aus einem laufenden Anwendungsprogramm, häufig mit der Möglichkeit der Beeinflussung der dargestellten Szenen durch Benutzerinteraktion



Realisierung:

- Programmbibliothek mit API
- spezielle Hardware (Graphikprozessen – Graphics Processing Units (GPUs)), die von der Programmbibliothek genutzt wird.

Grobstruktur aus Software-Sicht



Weit verbreitetes Szenenbeschreibungskonzept:

polygonbasierte Szenenrepräsentation

(1) Polygonbasierte Szenenrepräsentation =

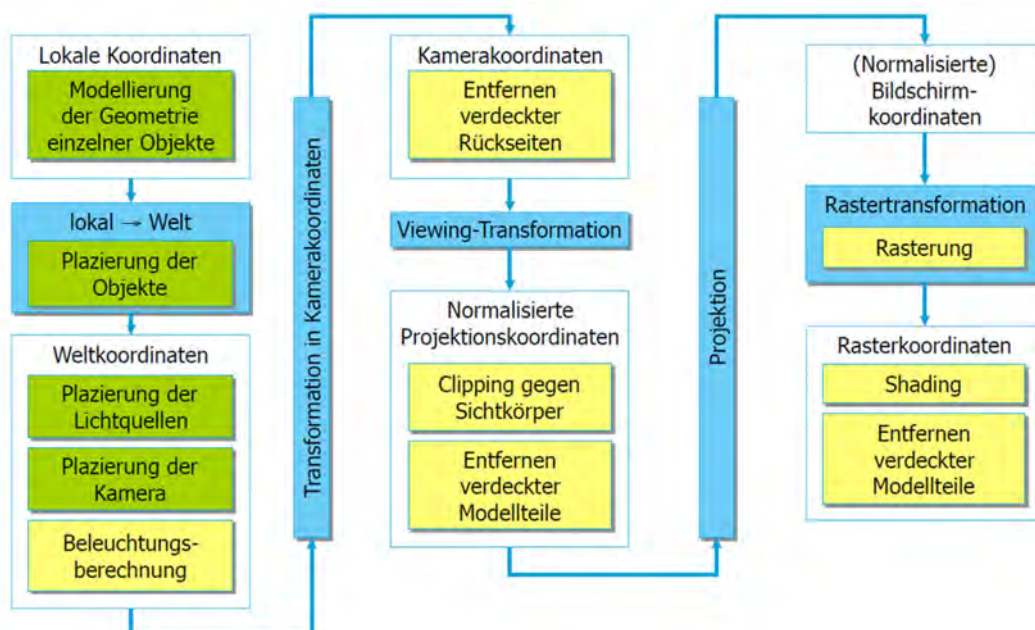
polygonbasierte Geometrirepräsentation
+ Materialrepräsentation

(2) Polygonbasierte Geometrirepräsentation =

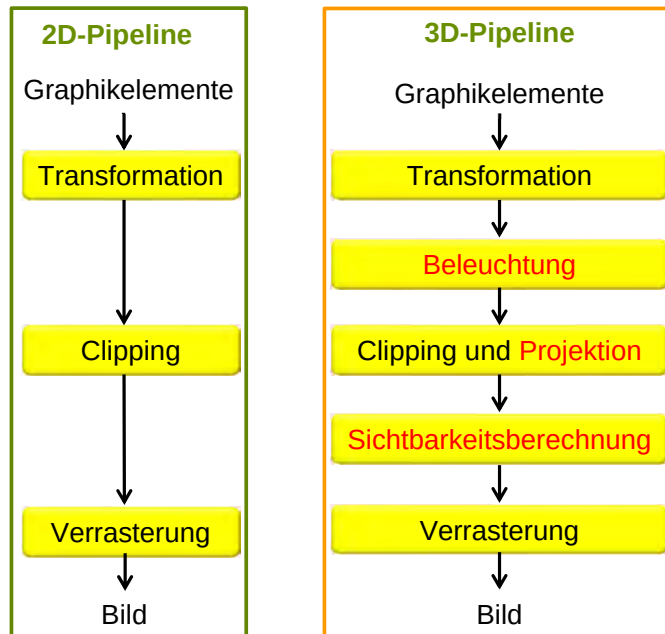
Menge von Punkten, Strecken oder Polygonen

(3) Materialrepräsentation =

Beleuchtungsmodell
+ Materialparameter des Beleuchtungsmodells

→ **Verarbeitung:** Stream (Strom) von Punkten, Strecken oder Polygonen**Zusammensetzen der Bilderzeugungs-Pipeline**

Vergleich zwischen den 2D- und 3D-Bilderzeugungs-Pipelines



Teilschritte der Bilderzeugungspipelines

1. Transformation:

Anwendung einer affinen Abbildung auf die Szene

Bsp.: Szene aus Dreiecken, Anwendung der affinen Abbildung, z.B. in homogener Darstellung, auf die Eckpunkte der Dreiecke

2. Beleuchtung:

Anwendung einer bidirektionalen Shading-Funktion (BSF) an Stützstellen der Szene, aus denen dann bei der Verrasterung durch Interpolation die Farbe berechnet wird.

Bsp.: Szene aus Dreiecken, Auswertung der BSF an den Eckpunkten, Interpolation durch Gouraud-Shading

3. Clipping:

Festlegung einer Sehpyramide, deren Inneres den Teil der Szene definiert, der im Bild dargestellt wird.



Teilschritte der Bilderzeugungs-pipelines

4. Projektion:

Projektion der dreidimensionalen Szene auf eine zweidimensionale Bildebene

Bsp.: perspektivische Projektion, Parallelprojektion

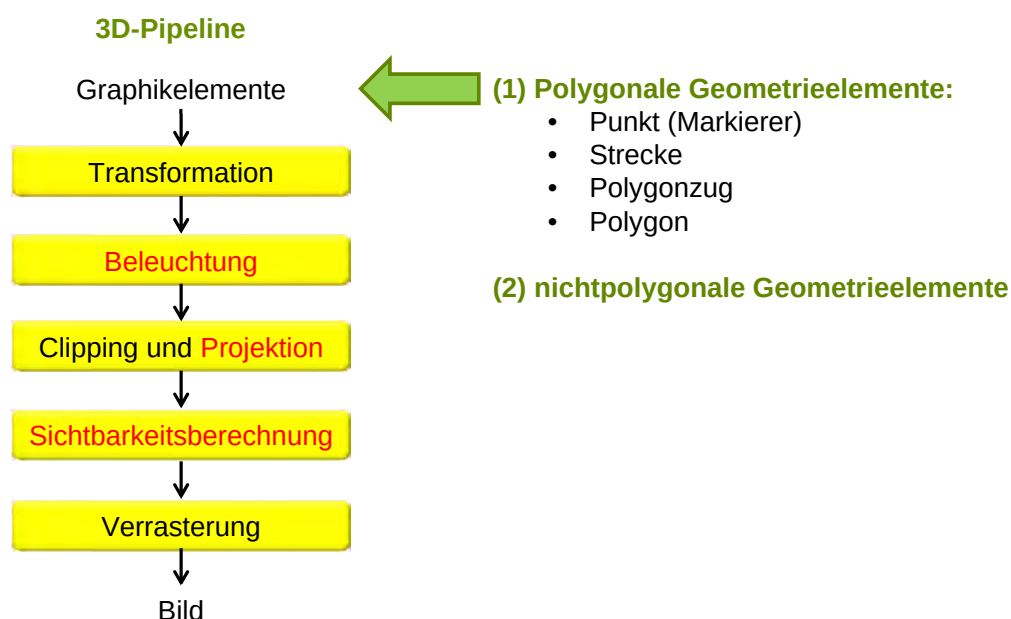
5. Sichtbarkeitsberechnung:

Entfernung der nicht sichtbaren Teile der Szene bezüglich eines Augenpunktes oder einer Blickrichtung

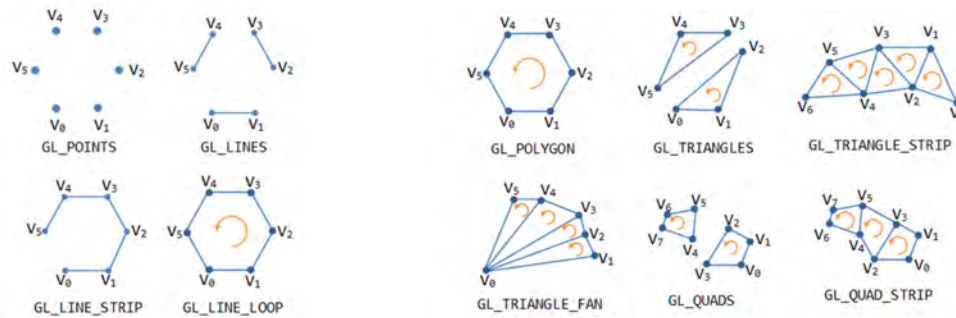
Bsp.: Sichtbarkeitsberechnung mit dem Tiefenpufferalgorithmus (z-buffer)

6. Verrasterung:

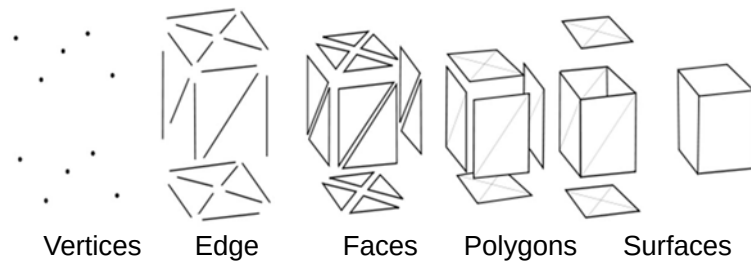
Umwandlung der gegebenen geometrischen Repräsentation der Szene in eine Rasterdarstellung. Manchmal wird dies schon zusammen mit der Sichtbarkeitsberechnung erledigt, z.B. beim Tiefenpufferalgorithmus.



2D-Graphikelemente (Bsp.: OpenGL → s.a. Kapitel „E. Computergraphik und OpenGL“)



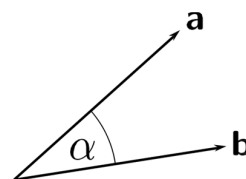
3D-Graphikelemente (Bsp.: Meshes)



Skalarprodukt (dot product)

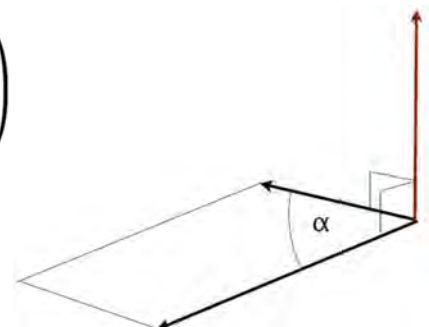
$$\mathbf{a} \cdot \mathbf{b} = a_x b_x + a_y b_y + a_z b_z = |\mathbf{a}| \cdot |\mathbf{b}| \cdot \cos \alpha$$

$$\cos \alpha = \frac{a_x b_x}{|\mathbf{a}| |\mathbf{b}|} + \frac{a_y b_y}{|\mathbf{a}| |\mathbf{b}|}$$

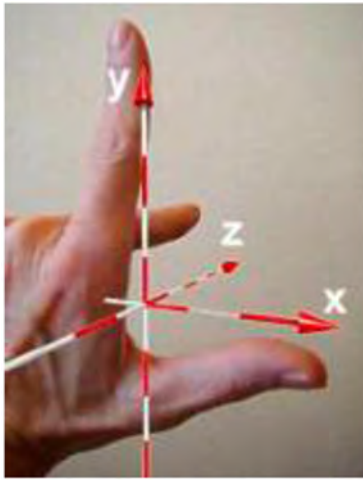


Kreuzprodukt (cross product)

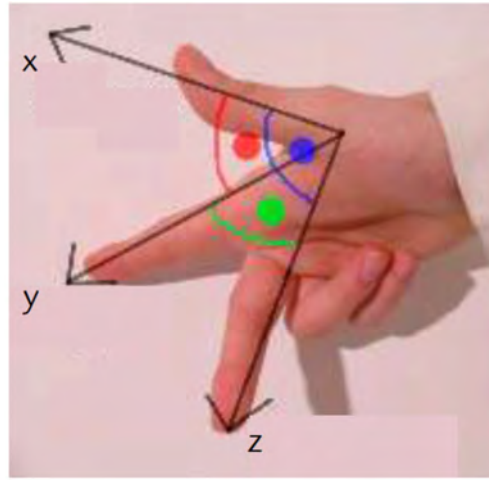
$$\mathbf{c} = \mathbf{a} \times \mathbf{b} = \begin{pmatrix} a_x \\ a_y \\ a_z \end{pmatrix} \times \begin{pmatrix} b_x \\ b_y \\ b_z \end{pmatrix} = \begin{pmatrix} a_y b_z - a_z b_y \\ a_z b_x - a_x b_z \\ a_x b_y - a_y b_x \end{pmatrix}$$



3D-Koordinatensysteme

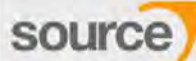


Left handed system
(Linkssystem)



Right handed system
(Rechtssystem)

bedingte Konvention

	Y-Up	Z-Up	
Right-handed	GODOT Game engine Houdini MARMOSET TOOLBAG	MAYA SUBSTANCE PAINTER MODO	CRYENGINE source blender 3DS MAX AUTOCAD SketchUp
Left-handed	unity CINEMA 4D	LightWave ZBRUSH	UNREAL ENGINE

Ebenen / Dreiecke

- Durch 3 Punkte wird eine Ebene aufgespannt
- Parameterdarstellung:

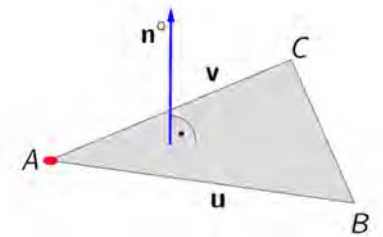
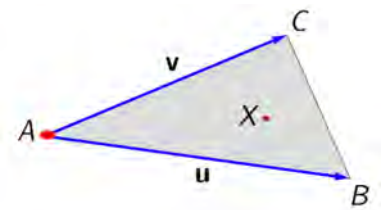
$$X = A + s \cdot \mathbf{u} + t \cdot \mathbf{v}$$

- Für Dreiecke gilt zusätzlich:

$$s, t \in (0, 1), \quad s + t \leq 1$$

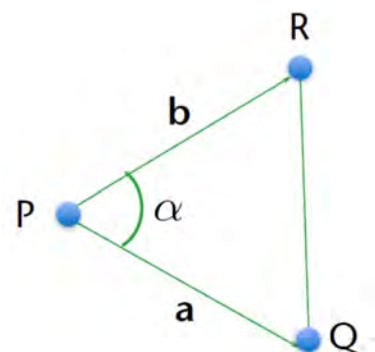
- Normale eines Dreiecks / einer Ebene:

$$\mathbf{n}^\circ = \frac{\mathbf{u} \times \mathbf{v}}{|\mathbf{u} \times \mathbf{v}|}$$



Flächeninhalt eines Dreiecks

$$\begin{aligned}
 A(\triangle PQR) &= \frac{1}{2} \|\mathbf{a}\| \cdot \|\mathbf{b}\| \sin \alpha \\
 &= \frac{1}{2} \|\mathbf{a} \times \mathbf{b}\| \\
 &= \begin{cases} \frac{1}{2} \|\mathbf{a} \times \mathbf{b}\|, & \text{gegen Uhrzeigersinn} \\ -\frac{1}{2} \|\mathbf{a} \times \mathbf{b}\|, & \text{Uhrzeigersinn} \end{cases} \\
 &= \frac{1}{2} \det \begin{pmatrix} P_x & P_y & 1 \\ Q_x & Q_y & 1 \\ R_x & R_y & 1 \end{pmatrix}
 \end{aligned}$$



Ebene

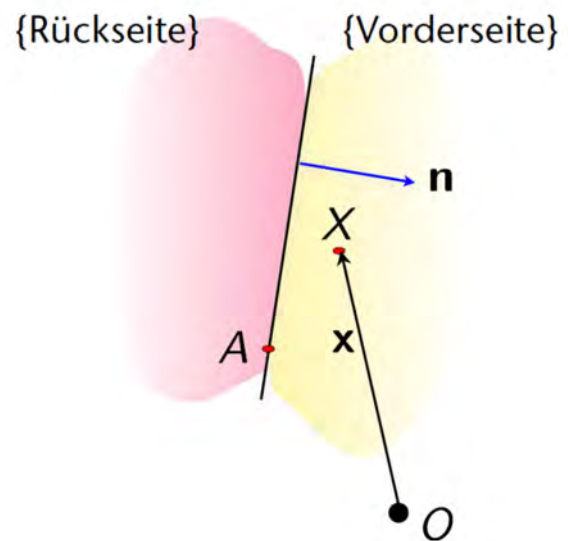
Eine Ebene $(\mathbf{n}, d)$ im $\mathbb{R}^k$ definiert

3 Äquivalenzklassen:

"Vorderseite" $:= \{X \mid \mathbf{x} \cdot \mathbf{n} - d > 0\}$

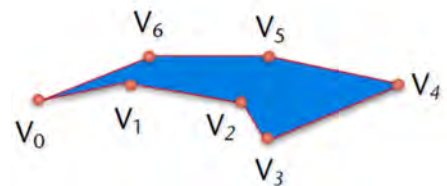
"Rückseite" $:= \{X \mid \mathbf{x} \cdot \mathbf{n} - d < 0\}$

Ebene selbst $:= \{X \mid \mathbf{x} \cdot \mathbf{n} - d = 0\}$

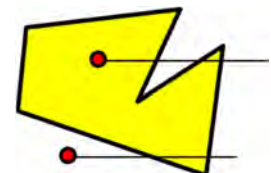


Polygone

Polygon ist ein geometrischer Graph $G=(V,E)$, mit den Knoten $V = \{v_0, v_1, \dots, v_{N-1}\} \in \mathbb{R}^d, d \geq 2$, und Kanten $e_{v_i, v_{i+1}}$, sodass alle Knoten in einer Ebene liegen.



Polygon: die Menge der Knoten der Ebene, für die ein dort ausgehender Strahl, der durch keinen der Knoten geht, den Polygonzug in einer ungeraden Anzahl von Punkten schneidet.



einfaches Polygon: der gegebene Polygonzug schneidet sich nicht selbst.

konvexes Polygon: ein einfaches Polygon, bei dem zu je zwei seiner Punkte auch die auf deren Verbindungsstrecke liegenden Punkte im Polygon liegen.

Konvexes: für alle $v_i, v_j \in G \rightarrow \text{gesamte Linie } \overline{v_i, v_j} \subseteq G$

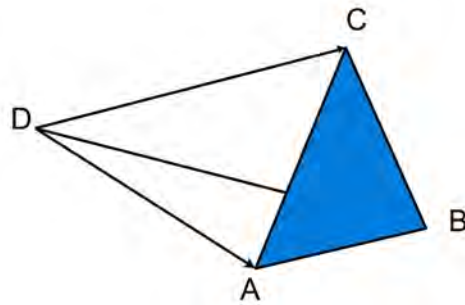
Umlaufsinn

Drei Punkte A, B, C erscheinen von einem vierten Punkt D aus entgegen dem Uhrzeigersinn $\Leftrightarrow$

$$(A - D) \times (B - D) \cdot (C - D) < 0$$

$$\Leftrightarrow \text{Vol}(DACB) < 0$$

$$\Leftrightarrow \text{Vol}(ABCD) < 0$$

**D.1. Grundkonzepte**

- D.1.1. Einführung
- D.1.2. Bilderzeugungs-Pipelines
- D.1.3. Geometrieprepräsentation
- D.1.4. Geometrieeigenschaften

D.2. Transformation

- D.2.1. Einleitung
- D.2.2. Affine Abbildungen
- D.2.3. Homogene Darstellung

D.3. Beleuchtungsberechnung

- D.3.1. Einleitung
- D.3.2. Beleuchtungsmodell von Phong
- D.3.3. Shading-Modelle

D.4. Clipping und Projektion

- D.4.1. Einleitung
- D.4.2. Clipping
- D.4.3. Picking
- D.4.4. Projektion
- D.4.5. Kamerakalibrierung

D.5. Sichtbarkeit

- D.5.1. Einleitung
- D.5.2. Szenenbasiertes Lösungsverfahren
 - D.5.2.1. Painter's Algorithm
 - D.5.2.2. Binäre Raumzerlegung
- D.5.3. Rasterbildbasierte Lösungsverfahren
 - D.5.3.1. Tiefenpufferverfahren
 - D.5.3.2. Scan-Line-Verfahren
 - D.5.3.3. Bereichsunterteilungsverfahren

D.6. Verrasterung

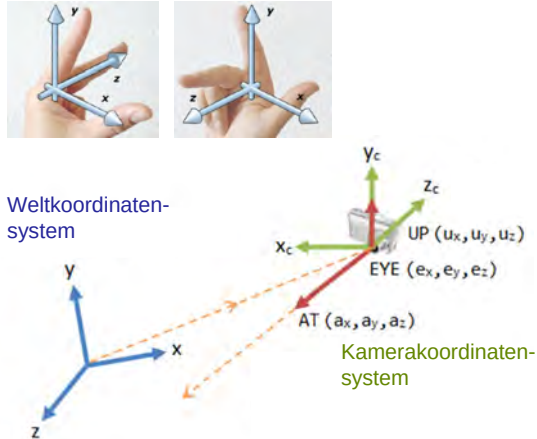
- D.6.1. Strecken in expliziter Darstellung
- D.6.2. Strecken in impliziter Darstellung
- D.6.3. Anti-Alias-Behandlung
- D.6.4. Dicke Strecken
- D.6.5. Verrasterung von Polygonen
- D.6.6. Beleuchtungsberechnung und Verrasterung
- D.6.7. Textur
- D.6.8. Flächenfüllen

Geometrische Transformationen: affine Abbildungen

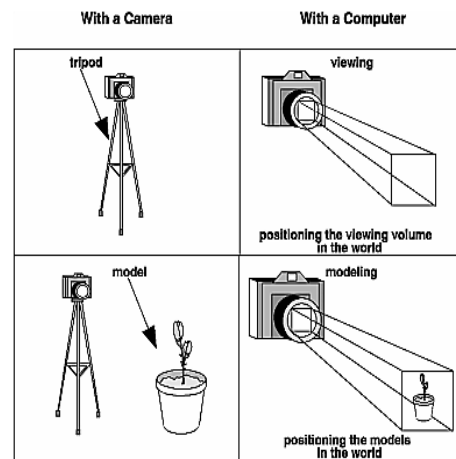
Motivation:

- Kamera- und Modellplatzierung
- bewegte Szenenelemente
- hierarchisch strukturierte Szenen

Koordinatensysteme:



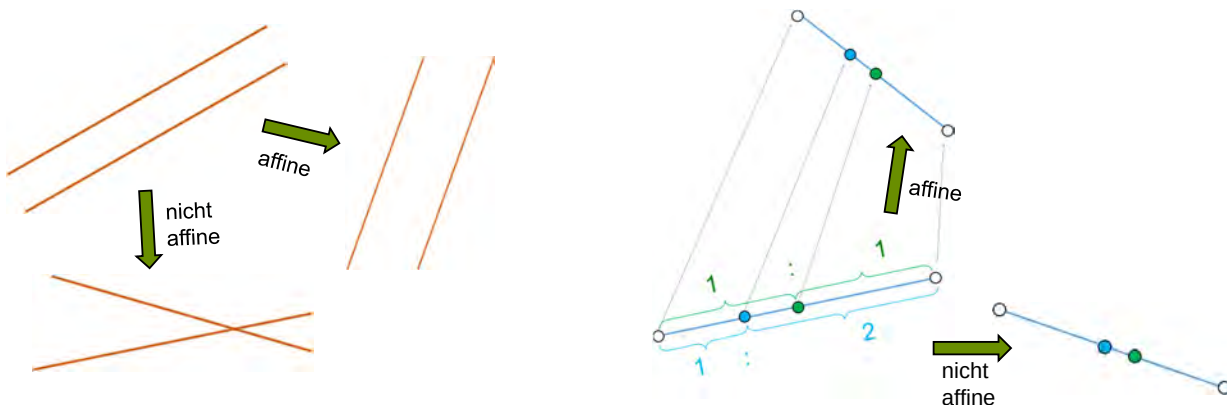
Bsp.: Kamera- und Modellplatzierung



Transformation: üblicherweise durch affine Abbildungen

Affine Abbildung (in der Ebene): $\bar{p} = A \cdot p + t$

mit $A = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix}$, $a_{ij} \in \mathbb{R}$, $t = \begin{pmatrix} t_1 \\ t_2 \end{pmatrix}$, $t_i \in \mathbb{R}$



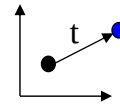
Transformation: üblicherweise durch affine Abbildungen

Affine Abbildung (in der Ebene): $\bar{p} = A \cdot p + t$

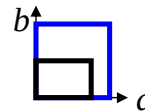
mit $A = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix}$, $a_{ij} \in \mathbb{R}$, $t = \begin{pmatrix} t_1 \\ t_2 \end{pmatrix}$, $t_i \in \mathbb{R}$

Spezialfälle für affine Abbildungen:

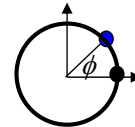
• **Translation:** $\bar{p} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \cdot p + t$.



• **Skalierung:** $\bar{p} = \begin{pmatrix} a & 0 \\ 0 & b \end{pmatrix} \cdot p$, $a, b \geq 0$.



• **Rotation:** $\bar{p} = \begin{pmatrix} \cos \phi & -\sin \phi \\ \sin \phi & \cos \phi \end{pmatrix} \cdot p$

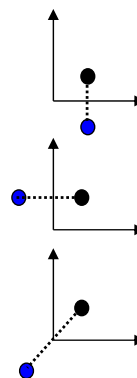


• **Spiegelung**

an der waagrechten Achse: $\bar{p} = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \cdot p$

an der senkrechten Achse: $\bar{p} = \begin{pmatrix} -1 & 0 \\ 0 & 1 \end{pmatrix} \cdot p$

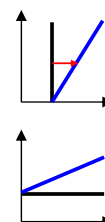
am Ursprung: $\bar{p} = \begin{pmatrix} -1 & 0 \\ 0 & -1 \end{pmatrix} \cdot p$



• **Scherung**

in Richtung der waagrechten Achse: $\bar{p} = \begin{pmatrix} 1 & s \\ 0 & 1 \end{pmatrix} \cdot p$

in Richtung der senkrechten Achse: $\bar{p} = \begin{pmatrix} 1 & 0 \\ s & 1 \end{pmatrix} \cdot p$





Spezialfälle im Raum:

Translation:

$$\bar{p} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \cdot p + t.$$

Skalierung:

$$\bar{p} = \begin{pmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & s_z \end{pmatrix} \cdot p, \quad a, b \geq 0.$$

Rotation:

um die x-Achse:

$$\bar{p} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \phi & -\sin \phi \\ 0 & \sin \phi & \cos \phi \end{pmatrix} \cdot p$$

um die y-Achse:

$$\bar{p} = \begin{pmatrix} \cos \phi & 0 & \sin \phi \\ 0 & 1 & 0 \\ -\sin \phi & 0 & \cos \phi \end{pmatrix} \cdot p$$

um die z-Achse:

$$\bar{p} = \begin{pmatrix} \cos \phi & -\sin \phi & 0 \\ \sin \phi & \cos \phi & 0 \\ 0 & 0 & 1 \end{pmatrix} \cdot p$$



Rotation mit Winkel ϕ um eine beliebige Achse:

1. Translation, sodass die Achse durch den Ursprung läuft;
2. Rotation um die z-Achse, sodass die Achse in der x-z-Ebene liegt;
3. Rotation um die y-Achse, sodass die Achse mit der z-Achse zur Deckung kommt;
4. Rotation um die z-Achse mit Winkel ϕ ;
5. Rücktransformation durch die inverse Transformation der ersten drei Schritte in umgekehrter Reihenfolge.



Homogene Darstellung affiner Abbildungen:

$$\bar{p}^* = A^* \cdot p^*$$

$$\text{mit } \bar{p}^* := \begin{pmatrix} \bar{p} \\ 1 \end{pmatrix}, p^* := \begin{pmatrix} p \\ 1 \end{pmatrix}, A^* := \begin{pmatrix} A & t \\ 0 & 1 \end{pmatrix},$$

wobei $\bar{p} = A \cdot p + t$ eine gegebene affine Abbildung ist.

Vorteil der homogenen Darstellung:

Hintereinanderausführung von Abbildungen durch einfache Matrizenmultiplikation



Hintereinanderausführung von Abbildungen

in homogener Darstellung durch einfache Matrizenmultiplikation

$$T_1 : \bar{p} = A_1 \cdot p + t_1,$$

$$T_2 : \bar{p} = A_2 \cdot p + t_2$$

$$\begin{aligned} \Rightarrow T_3 &:= T_2 \circ T_1 : \bar{p} = A_2 \cdot (A_1 \cdot p + t_1) + t_2 \\ &= A_2 \cdot A_1 \cdot p + A_2 \cdot t_1 + t_2 \\ &= A_3 \cdot p + t_3 \end{aligned}$$

$$\begin{aligned} \text{andererseits: } A_2^* \cdot A_1^* \cdot p^* &= \begin{pmatrix} A_2 \cdot A_1 & A_2 \cdot t_1 + t_2 \\ 0 & 1 \end{pmatrix} \cdot p^* \\ &= \begin{pmatrix} A_3 & t_3 \\ 0 & 1 \end{pmatrix} \cdot p^* \end{aligned}$$

Transformationsmatrix $T = \begin{pmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{pmatrix}$

Rotation
Skalierung
Translation
Homogene Erweiterung



Rotation: $\begin{pmatrix} \cos \alpha & -\sin \alpha & 0 \\ \sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 1 \end{pmatrix}$ Skalierung: $\begin{pmatrix} s_1 & 0 & 0 \\ 0 & s_2 & 0 \\ 0 & 0 & 1 \end{pmatrix}$ Translation: $\begin{pmatrix} 1 & 0 & a \\ 0 & 1 & b \\ 0 & 0 & 1 \end{pmatrix}$

Reflexion an der X-Achse: $\begin{pmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$ Reflexion an der Y-Achse: $\begin{pmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$ Reflexion an X-und Y-Achse: $\begin{pmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$

Hintereinanderausführung von Abbildungen durch

Transformationsmatrizen $T_1, T_2, \dots, T_N: \mathbf{p}' = T_N \cdot \dots \cdot T_2 \cdot T_1 \cdot \mathbf{p}$

← Reihenfolge der Ausführung

Affine Abbildung im 3D-Raum: $\bar{\mathbf{p}} = \mathbf{A} \cdot \mathbf{p} + \mathbf{t}$

mit $\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix}$, $a_{ij} \in \mathbb{R}$, $\mathbf{t} = \begin{pmatrix} t_1 \\ t_2 \\ t_3 \end{pmatrix}$, $t_i \in \mathbb{R}$

Homogene Darstellung affiner Abbildungen: $\bar{\mathbf{p}}^* = \mathbf{A}^* \cdot \mathbf{p}^*$

mit $\bar{\mathbf{p}}^* := \begin{pmatrix} \bar{\mathbf{p}} \\ 1 \end{pmatrix}$, $\mathbf{p}^* := \begin{pmatrix} \mathbf{p} \\ 1 \end{pmatrix}$, $\mathbf{A}^* := \begin{pmatrix} \mathbf{A} & \mathbf{t} \\ \mathbf{0} & 1 \end{pmatrix}$,

wobei $\bar{\mathbf{p}} = \mathbf{A} \cdot \mathbf{p} + \mathbf{t}$ eine gegebene affine Abbildung ist.

Transformationsmatrix $\mathbf{A}^* = \begin{pmatrix} a & b & c & d \\ e & f & g & h \\ i & j & k & l \\ 0 & 0 & 0 & 1 \end{pmatrix}$

Rotation
Skalierung
Translation
Homogene Erweiterung

D.1. Grundkonzepte

- D.1.1. Einführung
- D.1.2. Bilderzeugungs-Pipelines
- D.1.3. Geometrirepräsentation
- D.1.4. Geometrieigenschaften

D.2. Transformation

- D.2.1. Einleitung
- D.2.2. Affine Abbildungen
- D.2.3. Homogene Darstellung

D.3. Beleuchtungsberechnung

- D.3.1. Einleitung
- D.3.2. Beleuchtungsmodell von Phong
- D.3.3. Shading-Modelle

D.4. Clipping und Projektion

- D.4.1. Einleitung
- D.4.2. Clipping
- D.4.3. Picking
- D.4.4. Projektion
- D.4.5. Kamerakalibrierung

D.5. Sichtbarkeit

- D.5.1. Einleitung
- D.5.2. Szenenbasiertes Lösungsverfahren
 - D.5.2.1. Painter's Algorithm
 - D.5.2.2. Binäre Raumzerlegung
- D.5.3. Rasterbildbasierte Lösungsverfahren
 - D.5.3.1. Tiefenpufferverfahren
 - D.5.3.2. Scan-Line-Verfahren
 - D.5.3.3. Bereichsunterteilungsverfahren

D.6. Verrasterung

- D.6.1. Strecken in expliziter Darstellung
- D.6.2. Strecken in impliziter Darstellung
- D.6.3. Anti-Alias-Behandlung
- D.6.4. Dicke Strecken
- D.6.5. Verrasterung von Polygonen
- D.6.6. Beleuchtungsberechnung und Verrasterung
- D.6.7. Textur
- D.6.8. Flächenfüllen

D.3. Beleuchtungsberechnung

D.3.1. Einleitung

Ziel: Realistisch wirkende Darstellung dreidimensionaler Szenen durch Simulation optischer Effekte.



Alias Studio
(joshua.maruskadesign)



Visual Nature Studio



Open Inventor

GPU-Based Ray-Casting of
Quadratic Surfaces, Symp. on
Point-Based Graphics 06



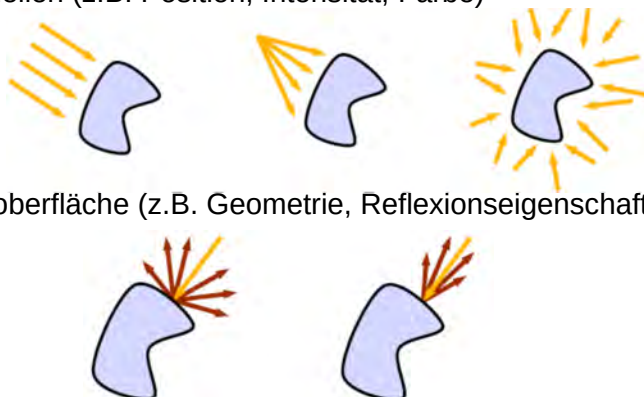
Ziel: Realistisch wirkende Darstellung dreidimensionaler Szenen durch Simulation optischer Effekte.



Impossible Rooftops (Kokichi Sugihara)

Beleuchtungsmodell = Konzept zur Berechnung der **Farb- und Helligkeitswerte** an Punkten auf der Oberfläche von Objekten

- Grundlage sind physikalische Gesetze
- Modellierung auf Basis verschiedener Einflüsse
 - 1) Lichtquellen (z.B. Position, Intensität, Farbe)
 - 2) Objektoberfläche (z.B. Geometrie, Reflexionseigenschaften)



Beleuchtungsmodell = Lichtquellenmodell
 + Reflexions- und Transmissionsmodell (Material)
 + Lichttransportmodell

Beleuchtung: geschieht durch lokale Reflexionsberechnung

Lichtquellen:

- punktförmig
- parallel einfallendes Licht (unendlich ferne Lichtquelle)
- ambientes Licht (ungerichtet)

Reflexion: festgelegt durch eine bidirektionale Shading-Funktion (BSF)

Lichttransport:

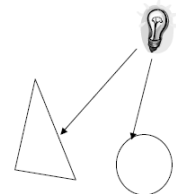
- Berücksichtigung von Verdeckungen bezüglich des Augenpunkts
- keine Berücksichtigung der Verdeckung von Lichtquellen (Schatten)
- nur Berücksichtigung der einmaligen Reflexion des von einer Lichtquelle einfallenden Lichts am Augenpunkt

Beleuchtungsmodelle:

beschreiben die Faktoren, welche die Farbe eines Objektes an einem bestimmten Punkt bestimmen

Lokale Beleuchtungsberechnung:

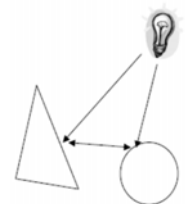
- Einschränkung: nur Interaktion Lichtquelle mit einem Objekt,
- integriert in das Pipeline-Konzept der Graphikhardware,
- oft heuristisch, approximierend
- einfacher als globale Beleuchtungsberechnung
- **Methoden:** Beleuchtungsmodell nach Phong und polygonales Shading



Kapitel E.

Globale Beleuchtungsberechnung:

- Einbeziehung aller Objekte einer Szene, um die Farbe an einem Punkt zu bestimmen,
- Interaktion Lichtquelle-Objekt und auch Interaktion Objekt-Objekt,
- oft physikalisch basiert (Erhaltungssätze)
- **Methoden:** Raytracing, Radiosity

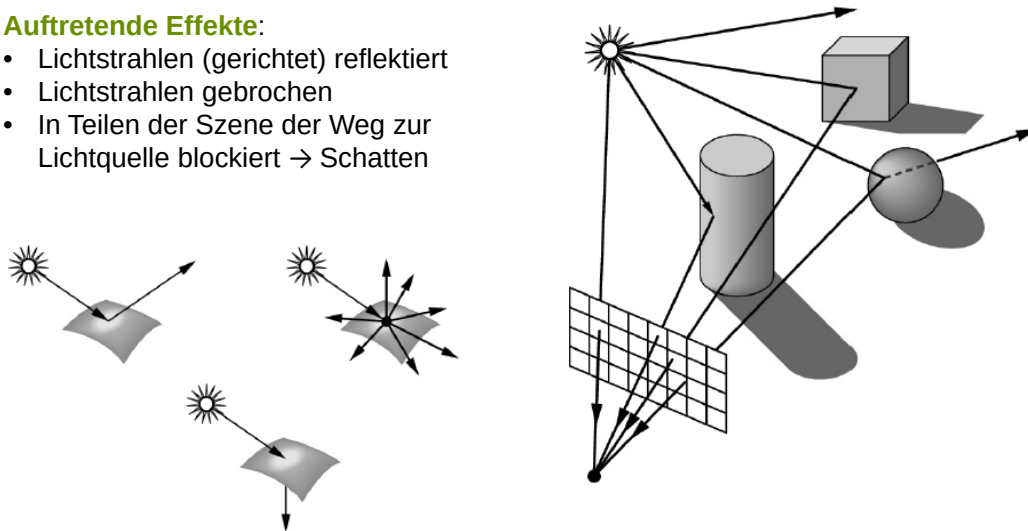


Ziel der Beleuchtungsmodelle:

Wechselwirkungen zwischen Licht und Oberflächen beschreiben, um Beleuchtungseffekte in das Rendering zu integrieren.

Auftretende Effekte:

- Lichtstrahlen (gerichtet) reflektiert
- Lichtstrahlen gebrochen
- In Teilen der Szene der Weg zur Lichtquelle blockiert → Schatten

**Beleuchtungsmodell von Phong:** $I(v) = I_a + I_d + I_g(v)$ **I_a : Ambienter Anteil**

- Grundhelligkeit in der Szene
- Simuliert Streuung des Lichtes durch Oberflächen (indirekte Beleuchtung)
- Unabhängig von Betrachterstandpunkt und vom einfallendem Licht

 I_d : Diffuser Anteil

- Reflexion an matten Oberflächen
- Gleichmäßig in alle Richtungen
- Unabhängig vom Betrachterstandpunkt

 I_g : Glänzender (spekularer) Anteil

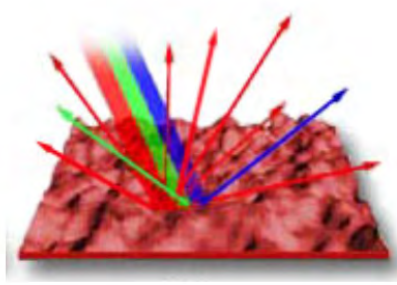
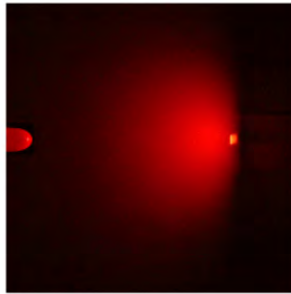
- Gerichtete Reflexion an spiegelnden Oberflächen
- Abhängig vom Betrachterstandpunkt
- Erzeugt Glanzpunkte

Bemerkung:

$I(\lambda, v) = \sum_l I_l(\lambda, v)$ wäre korrekt. Zur Vereinfachung wird λ im Folgenden weggelassen

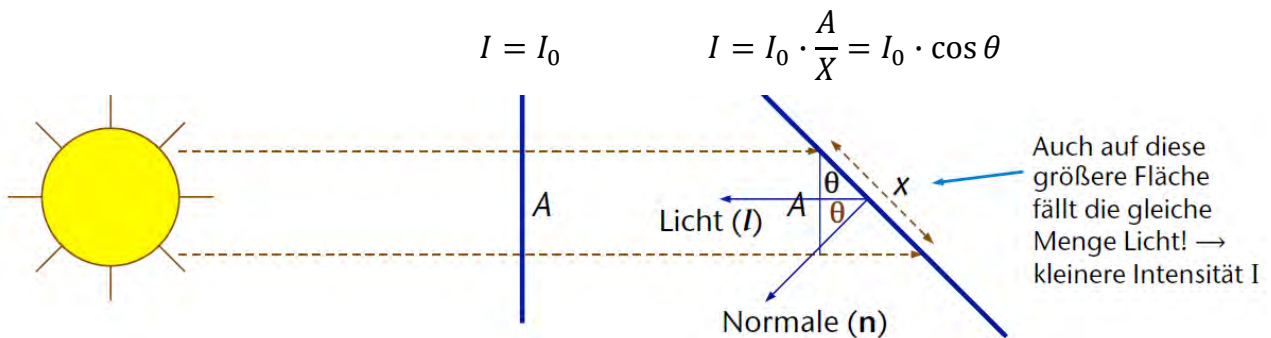
Diffuser Reflexion

- Licht wird von der Objektoberfläche gleichmäßig in alle Richtungen reflektiert
- Folge: Helligkeit ist unabhängig vom Sichtbereich
- Beispiele: Stück Papier, Tafel, unbearbeitetes Holz
- Diffuse/Matte Objekte werden auch Lambert'sche Objekte bezeichnet

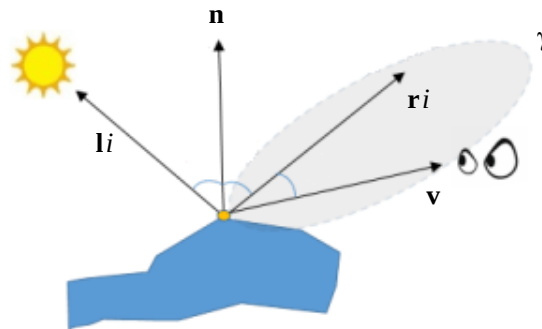
**Lambert'schen Kosinus-Gesetz**

Lichtstärke I_0 eines flächenhaften Strahls variiert mit dem Kosinus des Winkels zur Normalen n . Da der Mensch jedoch mit dem Auge nur die Leuchtdichte L wahrnehmen kann, erscheint der bestrahlte Körper dennoch unabhängig vom Betrachtungswinkel als gleich hell: $I = L \cdot A \cdot \cos \theta$.

$$I = I_0 \cdot (n \cdot l) = I_0 \cdot \cos \theta$$



Beleuchtungsmodell von Phong: $I(v) = I_a + I_d + I_g(v)$



Geometrische Parameter:

- v normierte Blickrichtung
- n auf Länge 1 normierte Oberflächennormale
- l_i auf Länge 1 normierte Richtungsvektor zur i -ten Lichtquelle
- r_i auf Länge 1 normierte Reflexionsvektor zu l_i nach dem Reflexionsgesetz
- $\gamma > 0$ Ausdehnung des Glanzlichtes

 Demo zum Phong-Modell

• **ambiante Reflexion**

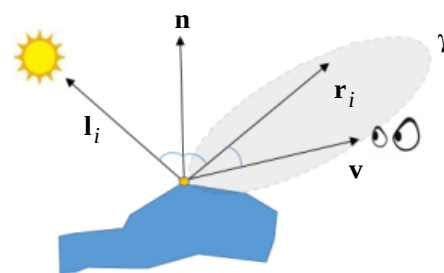
$$I_a := k_a \cdot I_{amb},$$

• **diffuse Reflexion**

$$I_d := k_d \sum_{i=1}^l n * l_i \cdot I_i / f(d_i)$$

• **glänzende Reflexion**

$$I_g := k_g \sum_{i=1}^l (v * r_i)^\gamma \cdot I_i / f(d_i).$$



$k_a, k_d, k_g, \gamma > 0$

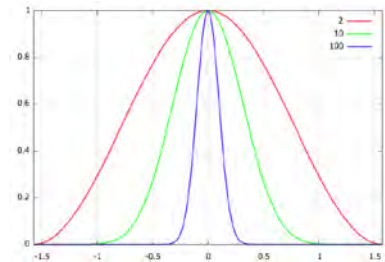
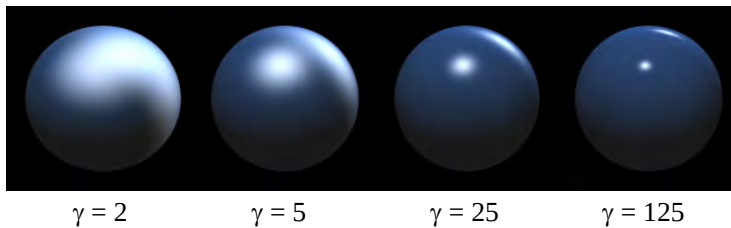
Bemerkung:

- I_i : Intensität der i -ten Lichtquelle.
- „*“: Transponierung des Vektor; Ausdrücke in den Formeln somit Skalarprodukte, ergeben den Kosinus des Winkels zw. jeweils multiplizierten normierten Vektoren.
- k_a, k_d, k_g : Intensitäten/Koeffizienten mit Farbkomponenten *Rot*, *Grün* und *Blau*
- d_i : Entfernung zur i -ten Lichtquelle, $f(.)$: Funktion des Abstands

Beleuchtungsmodell von Phong: $I(\mathbf{v}) = I_a + I_d + I_g(\mathbf{v})$

$$I(\mathbf{v}) = \mathbf{k}_a \cdot \mathbf{I}_{amb} + \sum_{i=1}^l \mathbf{I}_i / f(d_i) \cdot [\mathbf{k}_d(\mathbf{n} * \mathbf{l}_i) + \mathbf{k}_g(\mathbf{v} * \mathbf{r}_i)^\gamma]$$

Variation des Termin γ zum Glanzlicht



Schattenberechnung

Schattenterm S_i in Beleuchtungsformel $I(\mathbf{v})$ einsetzen:

$$I(\mathbf{v}) = \mathbf{k}_a \cdot \mathbf{I}_{amb} + \sum_{i=1}^l S_i \cdot \mathbf{I}_i / f(d_i) \cdot [\mathbf{k}_d(\mathbf{n} * \mathbf{l}_i) + \mathbf{k}_g(\mathbf{v} * \mathbf{r}_i)^\gamma]$$

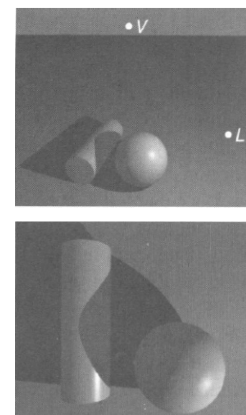
$$\text{mit } S_i = \begin{cases} 0 & \text{wenn Licht } i \text{ an diesem Punkt blockiert} \\ 1 & \text{sonst} \end{cases}$$

Realisierung:

1. Verwendet analytischen Sichtbarkeitsalgorithmus
2. Pixelbasiert (Shadow-Map)
 - Berechnet z-Buffer (s. Kapitel D.9. Sichtbarkeit) von Betrachter und Lichtquelle aus

Algorithmus:

- Für alle Pixel des Bildes
 - Bestimme Koordinaten in der Lichtquelle (x,y,z)
 - Wenn Tiefe größer als Wert im z-Buffer der Lichtquelle blockiere den Punkt (Schatten)



Beleuchtungsmodell = Lichtquellenmodell
 + Reflexions- und Transmissionsmodell (Material)
 + Lichttransportmodell

Beleuchtungsberechnung für Polygone/Polygonszenen

Ziel: Zuordnung von Farben zu den Pixeln

- Betrachten polygonale 3D-Modelle.
- Verfahren unterscheiden sich darin, an welchen Stellen das Beleuchtungsmodell ausgewertet wird.

Einfache Techniken: interpolative Shading-Verfahren

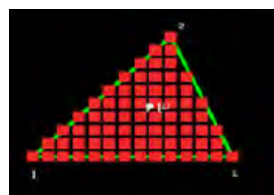
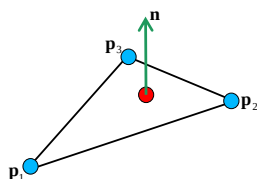
- Über das Beleuchtungsmodell werden Farbwerte an den Eckpunkten berechnet.
- Farbwerte aller Punkte eines Polygons ergeben sich durch lineare Interpolation der Farbwerte an den Eckpunkten.

Verfahren

- Flat-Shading
- Gouraud-Shading
- Phong-Shading

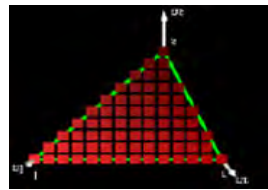
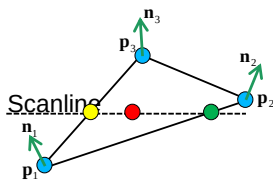
Flat Shading:

- Keine Interpolation, bestimme einen Farbwert pro Polygon
 - o Variante 1: Mittelwert aus den Farbwerten an den Eckpunkten, nach dem Beleuchtungsmodell
 - o Variante 2: Auswahl eines Punktes und Berechnung der Farbe an dem Punkt
- Die vom Polygon überdeckten sichtbaren Pixel werden mit dem berechneten Farbwert gefärbt.
- Einzelne Polygone deutlich sichtbar
- Einfache Implementierung, sehr schnell, aber geringe Qualität

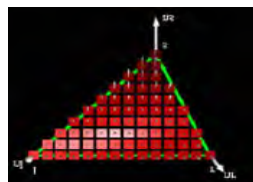
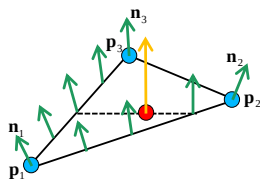


Gouraud-Shading („Smooth Shading“):

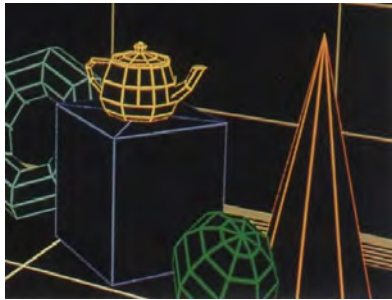
- Berechnen der Farbwerte an den Ecken des Polygons nach dem Beleuchtungsmodell
- Lineare Interpolation der Farbwerte von den Ecken in das Innere des Polygons
- Aufwändiger als Flat-Shading, aber qualitativ besser
- Kanten zwischen Polygonen nicht mehr sichtbar
- Problem: Glanzpunkt in der Mitte eines Polygons

**Phong-Shading:**

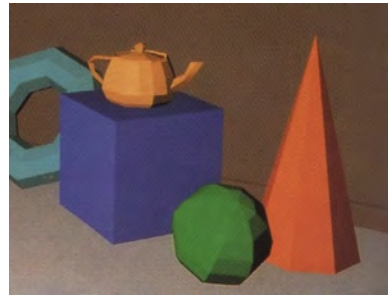
- Material- und Normalenattribute für die Polygone-Eckpunkte
- Interpoliere die Normalenvektoren der Eckpunkte des Polygons über die Fläche des Polygons oder Schätzungen davon
- Berechne an jedem Punkt (Pixel) im Inneren des Polygons einen Farbwert aus der interpolierten Normalen
- Aufwändiger als Gouraud-Shading, aber die besten Ergebnisse
- Glanzpunkte im Inneren eines Polygons werden korrekt dargestellt
- Aber: auch einige Fälle, die nicht dargestellt werden können



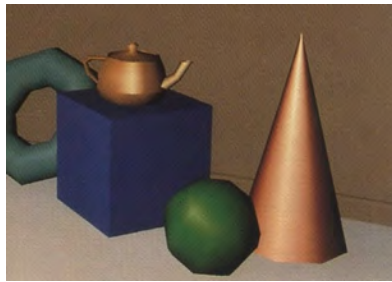
Beispiele:



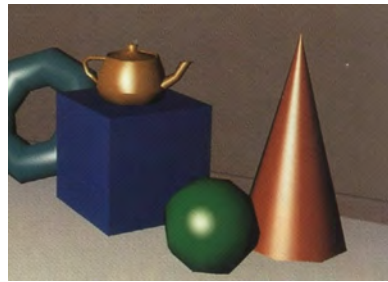
Originalszene aus glatten Objekten



Flat Shading der polygonapprox. Objekte



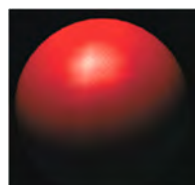
diffuses Smooth Shading der Polygonappr. glänzendes Smooth Shading der Polygonappr.



Vergleich zwischen Flat-, Gouraud- und Phong-Shading



Flat

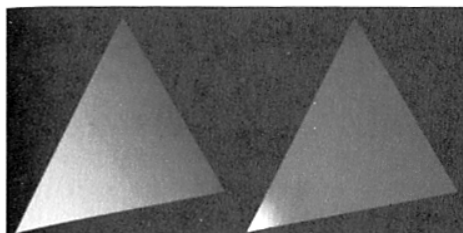


Gouraud



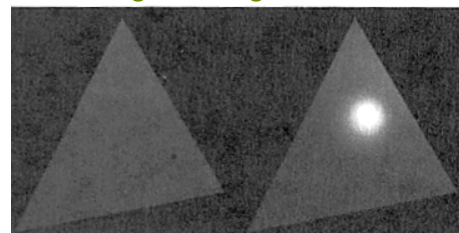
Phong

Glanzreflektion mit Gouraud-Shading und Phong-Shading



Gouraud-Shading Phong-Shading

Glanzlicht am linken Knoten



Gouraud-Shading Phong-Shading

Glanzlicht im Inneren des Dreiecks

D.1. Grundkonzepte

- D.1.1. Einführung
- D.1.2. Bilderzeugungs-Pipelines
- D.1.3. Geometrierepräsentation
- D.1.4. Geometrieeigenschaften

D.2. Transformation

- D.2.1. Einleitung
- D.2.2. Affine Abbildungen
- D.2.3. Homogene Darstellung

D.3. Beleuchtungsberechnung

- D.3.1. Einleitung
- D.3.2. Beleuchtungsmodell von Phong
- D.3.3. Shading-Modelle

D.4. Clipping und Projektion

- D.4.1. Einleitung
- D.4.2. Clipping
- D.4.3. Picking
- D.4.4. Projektion
- D.4.5. Kamerakalibrierung

D.5. Sichtbarkeit

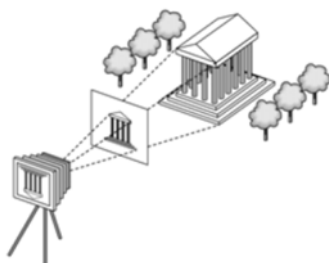
- D.5.1. Einleitung
- D.5.2. Szenenbasiertes Lösungsverfahren
 - D.5.2.1. Painter's Algorithm
 - D.5.2.2. Binäre Raumzerlegung
- D.5.3. Rasterbildbasierte Lösungsverfahren
 - D.5.3.1. Tiefenpufferverfahren
 - D.5.3.2. Scan-Line-Verfahren
 - D.5.3.3. Bereichsunterteilungsverfahren

D.6. Verrasterung

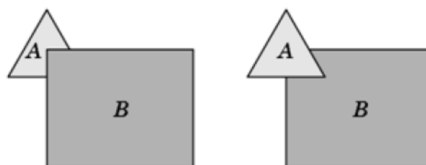
- D.6.1. Strecken in expliziter Darstellung
- D.6.2. Strecken in impliziter Darstellung
- D.6.3. Anti-Alias-Behandlung
- D.6.4. Dicke Strecken
- D.6.5. Verrasterung von Polygonen
- D.6.6. Beleuchtungsberechnung und Verrasterung
- D.6.7. Textur
- D.6.8. Flächenfüllen

Clipping:

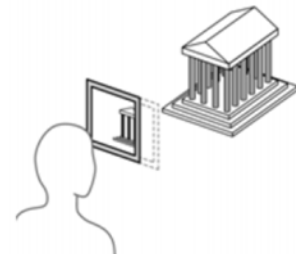
Abschneiden von Teilen einer Szene, die außerhalb eines gegebenen Bereichs liegen.



a.) Clipping am zu
sehenden Objekt



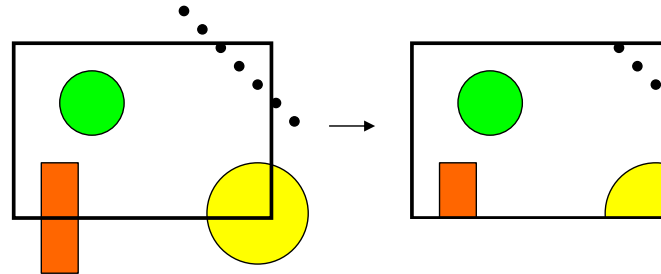
b.) Clipping bei
Verdeckung



c.) Clipping am
Viewport

Clipping:

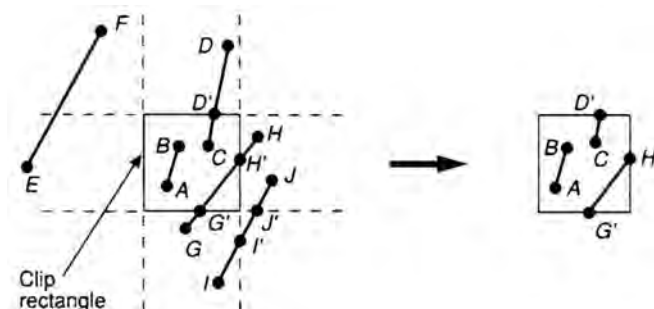
Abschneiden von Teilen einer Szene, die außerhalb eines gegebenen Bereichs liegen.

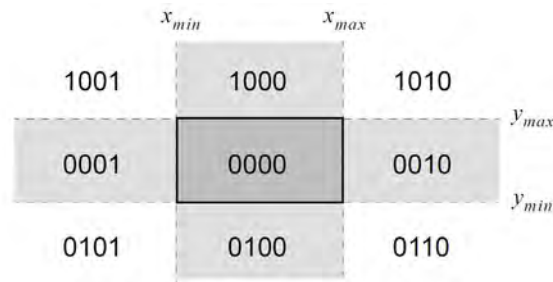
**Beispiele:**

- Punkt-Clipping
- Strecken-Clipping (z.B. Cohen-Sutherland)
- Polygon-Clipping (z.B. Sutherland-Hodgman)
- Kreis-Clipping

Strecken-Clipping (z.B. Cohen-Sutherland)**Idee**

1. Schritt: Zur Vermeidung der rechenaufwendigen Schnittpunktberechnungen wird versucht, möglichst viele Linien durch einfache Vergleiche zu akzeptieren oder zu eliminieren.
2. Schritt: Sofern Schritt 1 nicht gelingt, wird die Linie in zwei Segmente unterteilt, wobei eines jeweils einfach behandelt werden kann.



Strecken-Clipping (z.B. Cohen-Sutherland):**1. Schritt: Sektionierung der Ebene über 4-Bit-Code mit Vorzeichen****Bedeutung der Bits:**

1. Bit: In der Halbebene über der oberen Kante $y > y_{max}$
2. Bit: In der Halbebene unter der unteren Kante $y < y_{min}$
3. Bit: In der Halbebene rechts der rechten Kante $x > x_{max}$
4. Bit: In der Halbebene links der linken Kante $x < x_{min}$

Strecken-Clipping (z.B. Cohen-Sutherland):**Vorzeichenberechnung der Bits:**

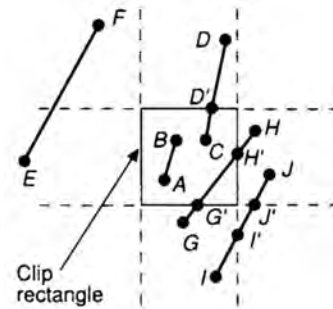
1. Bit: $y_{max} - y$
2. Bit: $y - y_{min}$
3. Bit: $x_{max} - x$
4. Bit: $x - x_{min}$

Codeberechnung:

- Für jeden Linienendpunkt Code berechnen
- Anschließend werden Codes beider Linienendpunkte binär **AND**-verknüpft
- Fallunterscheidung:
 - a) Falls $code1 \wedge code2 \neq 0$ kann die Linie entfernt werden, weil kein Schnitt mit dem Clipping-Rechteck besteht.
 - b) Für $code1 \vee code2 = 0$ liegt die Linie ganz innerhalb des Clipping-Rechtecks.

Strecken-Clipping (z.B. Cohen-Sutherland)**Beispiel zum 1. Schritt:**

Ausgangslage

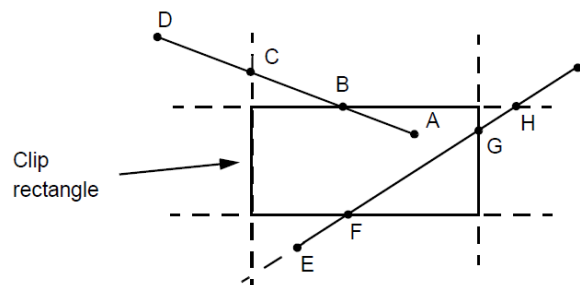


Kodierung nach dem 1. Schritt

Linie	code ₁	code ₂	code ₁ $\wedge$ code ₂
A $\rightarrow$ B	0000	0000	0000
C $\rightarrow$ D	0000	1000	0000
E $\rightarrow$ F	0001	1001	0001
G $\rightarrow$ H	0100	0010	0000
I $\rightarrow$ J	0100	0010	0000

Strecken-Clipping (z.B. Cohen-Sutherland):**2. Schritt: Iterative Subdivision**

Anhand der Codes der Linien-Endpunkte und der im 1. Schritt durchgeführten Tests Schnitte zwischen den Kanten und dem Clipping-Rechteck bestimmen.

Beispiel zum 2. Schritt:

Beispiel:

Da Punkt D durch 1001 kodiert, wird obere und linke Kante geschnitten

Strecken-Clipping (z.B. Cohen-Sutherland):

Verbleibende Linien werden gemäß ihrer Schnittpunkte mit den Rechteckkanten über eine feste Ordnung bei der Interpretation der Bits im Code (z. B. oben → unten → rechts → links) unterteilt.

Iteratives Vorgehen, bis die Linie trivial akzeptiert wird:

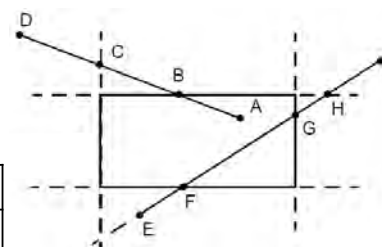
- 1) Auswahl eines der beiden Punkte der Linie. Der Punkt muss in der äußeren Halbebene einer Rechteckkante liegen.
- 2) Unterteilen der Linie in zwei Segmente an ihrem Schnittpunkt mit der Rechteckkante höchster Priorität
- 3) Eliminieren des Segments Punkt→Schnittpunkt
- 4) Berechnung des Codes für den Schnittpunkt

Strecken-Clipping (z.B. Cohen-Sutherland):**Beispiel:**

- Punkt *D* als Startpunkt wählen und wegen der Prioritätskonvention Schnittpunkt *B* berechnet. *D*→*B* wird eliminiert und *B*→*A* akzeptiert.
- Für Linie *E*→*I* wird zunächst *E* (0100) ausgewählt und Schnittpunkt *H* (0010) Restsegment *E*→*H* berechnet. Für dieses wird *E* als Startpunkt gewählt und aufgrund der Prioritäten zunächst *F* berechnet. Schließlich erfolgt die Berechnung der geclippten Linien aus *F*→*H* durch Teilung in *G*.

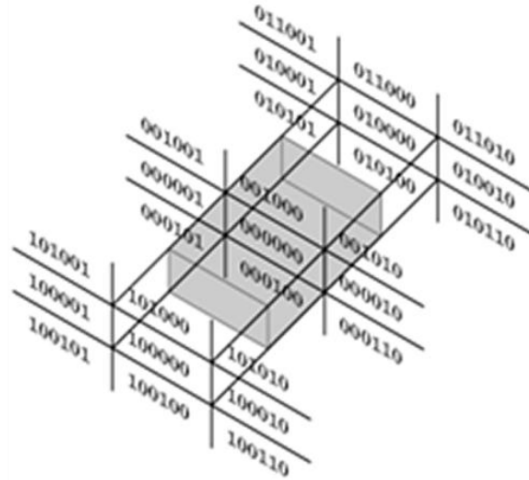
	x_{min}	x_{max}	
	1001	1000	1010
	0001	0000	0010
	0101	0100	0110
			y_{max}
			y_{min}

Linie	code ₁	code ₂	code ₁ ∧ code ₂	code ₁ ∨ code ₂
D→B	1001	0000	0000	1001
B→A	0000	0000	0000	0000



Strecken-Clipping (z.B. Cohen-Sutherland):**Clipping im 3D**

Erweiterung des Cohen-Sutherland-Algorithmus durch Ergänzen der Bitfelder pro Eckpunkt (zwei zusätzliche Bits repräsentieren Vergleich mit $z_{\min}$ und $z_{\max}$)

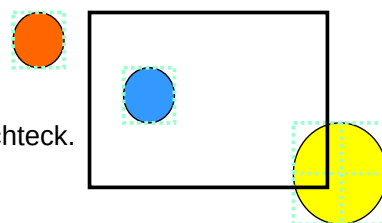
**Kreis-Clipping**

Eingabe: Ein Kreis, ein Rechteck.

Ausgabe: Der Durchschnitt des Kreises mit dem Rechteck.

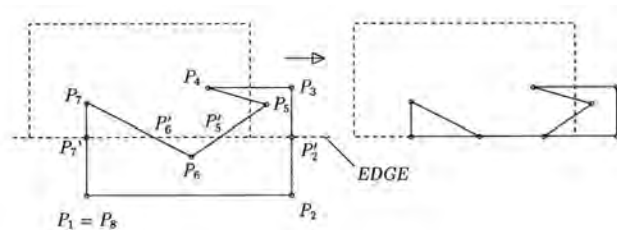
Ablauf:

- (1) Teste das achsenparallele Hüllquadrat des Kreises auf Schnitt mit dem Clipping-Rechteck;
- (2a) Wenn ein Schnitt festgestellt wird, teile den Kreis in vier Quadranten und führe den Test für jeden Viertelkreis aus;
- (2b) Wenn ein Schnitt festgestellt wird, berechne die Schnittpunkte mit den in Frage kommenden Rechteckseiten und bestimme die im Inneren des Clipping-Rechtecks liegenden Kreisabschnitte;
- (2c) Andernfalls liegt der Kreis ganz im Inneren, ganz im Äußeren des Clipping-Rechtecks oder umfasst das Clipping-Rechteck.

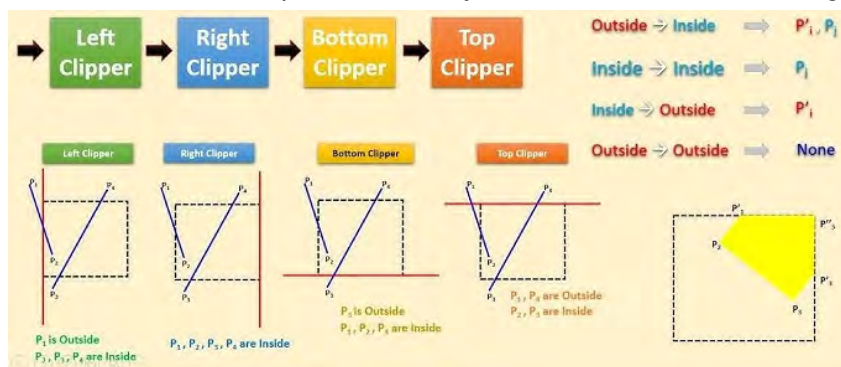


Polygon-Clipping**Algorithmus** Sutherland-Hodgman**Gegeben:** Ein Polygonzug, ein Rechteck**Gesucht:** Der Durchschnitt des Polygonzugs mit dem Rechteck**Ablauf:**

- Führe für die Geraden, welche die vier Rechteckseiten definieren, nacheinander aus:
- entferne die Eckpunkte des Polygonzugs, die in der äußeren Halbebene liegen und füge für jede Polygonkante, welche die Gerade schneidet, den entsprechenden Schnittpunkt in den Polygonzug ein.

**Polygon-Clipping****Algorithmus** Sutherland-Hodgman**Bemerkung:**

1. Der obige Algorithmus wird beim Clipping gegen ein Rechteck vier Mal hintereinander ausgeführt, wobei das Resultat eines Schritts die Eingabe des nächsten Schrittes bildet. In Hardware kann dies durch eine Pipeline aus vier Prozessoren realisiert werden.
2. Test einer Strecke auf Schnitt mit einer waagrecht begrenzten Halbebene: Vergleich der y-Koordinaten der Streckeneckpunkte mit der y-Koordinate der Halbebenengerade



Picking:

Auffinden eines geometrischen Objekts durch Identifizieren eines seiner Punkte.

Beispiel:

Anfahren von Objekten auf dem Bildschirm mit dem Cursor.

Schwierigkeiten:

1. die exakte Punktauswahl beim Picking „dünner“ Objekte, z.B. beim Picken von Punkten und Linien
2. das Identifizieren eines Objektpunktes, z.B. im Inneren eines Polygons.

Bemerkung:

Bei vielen geometrischen Objekten kann als Vortest auch zunächst das kleinste achsenparallele umfassende Rechteck für den Test verwendet werden.

Die so nicht ausgeschiedenen Objekte werden anschließend genauer untersucht.

Lösung zu 1:

Ein Objekt wird identifiziert, wenn der Auswahlpunkt hinreichend dicht bei ihm liegt oder das Objekt das dem Auswahlpunkt am nächsten liegende ist.

Entfernungsmaße zwischen zwei Punkten:

- Euklidischer Abstand (sehr rechenaufwendig)

$$d = \sqrt{(q_x - p_x)^2 + (q_y - p_y)^2}$$

Zum Test der relativen Entfernung braucht die Wurzel nicht gezogen werden, da mit $d_1 < d_2$ auch $d_1^2 < d_2^2$.

- Alternative:

$$d = (|q_x - p_x| + |q_y - p_y|) \text{ oder } d = \max\{|q_x - p_x|, |q_y - p_y|\}$$

Lösung zu 2: für Polygone

Zählen der Schnittpunkte auf einem Strahl:

Ein Punkt liegt genau dann im Polygon, wenn die **Anzahl der Wechsel** zwischen Innerem und Äußeren des Polygons eines von ihm ausgehenden Strahls **ungerade** ist:

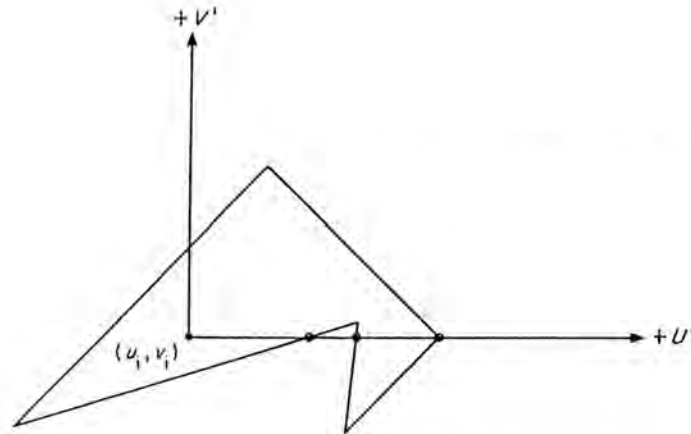
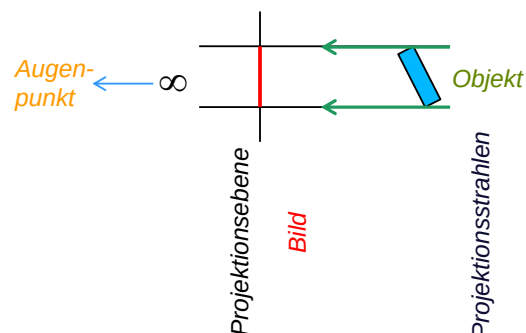
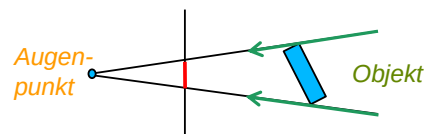
**Projektion:**

Abbildung aus einem Raum der Dimension n (hier: 3D) in einen Raum der Dimension $m < n$ (hier: 2D):

- Parallelprojektion**



- perspektivische Projektion**



Demo zur Projektion

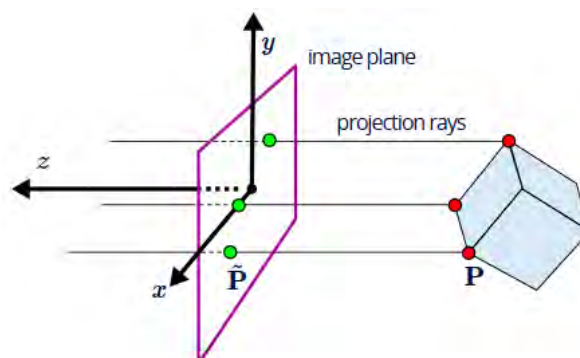
Lochkamera

- Computergraphik: idealisiertes Kameramodell mit einem unendlich kleinen Loch
- keine Tiefenunschärfe, d.h. alle Objekte werden scharf abgebildet
- Bild auf einer imaginären Bildebene vor dem Projektionszentrum, sodass das Bild nicht mehr auf dem Kopf steht

**Parallelprojektion**

- Projektionsstrahlen sind parallel, d.h. das Projektionszentrum liegt im Unendlichen
- Abbildung eines 3D-Punktes $\mathbf{P} = (p_x, p_y, p_z)^\top$ auf einen Punkt $\tilde{\mathbf{P}} = (\tilde{p}_x, \tilde{p}_y, \tilde{p}_z)^\top$, der in der Bildebene der Kamera liegt, ist gegeben durch:

$$\tilde{\mathbf{P}} = (p_x, p_y, 0)^\top$$



Parallelprojektion

- Bei Verwendung homogener Koordinaten kann die Parallelprojektion als lineare Abbildung mit einer 4×4-Matrix geschrieben werden:

$$\tilde{\mathbf{P}} = \begin{pmatrix} p_x \\ p_y \\ 0 \\ 1 \end{pmatrix} = \underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}}_{\mathbf{A}} \begin{pmatrix} p_x \\ p_y \\ p_z \\ 1 \end{pmatrix}$$

$$\tilde{\mathbf{P}} = \mathbf{A} \mathbf{P}$$

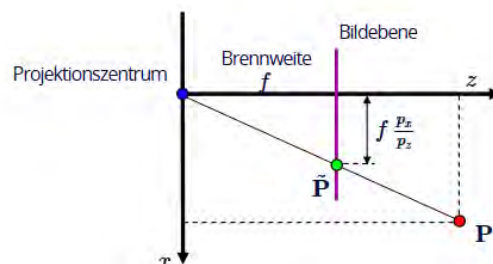
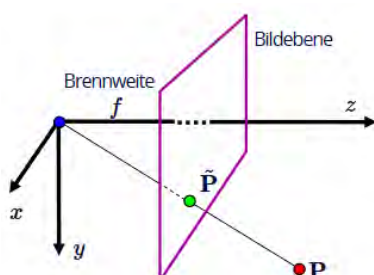
Perspektivische Projektion

- Projektionsvorschrift zur Abbildung eines 3D-Punkts $\mathbf{P} = (p_x, p_y, p_z)^\top$ auf einen Punkt $\tilde{\mathbf{P}} = (\tilde{p}_x, \tilde{p}_y, \tilde{p}_z)^\top$ auf der Bildebene der Kamera:

$$\tilde{\mathbf{P}} = \left(f \frac{p_x}{p_z}, f \frac{p_y}{p_z}, f \right)^\top$$

- leitet sich mit Hilfe des Strahlensatzes ab:

$$\frac{\tilde{p}_z}{f} = \frac{p_x}{p_z} \text{ und } \frac{\tilde{p}_y}{f} = \frac{p_y}{p_z}$$



Perspektivische Projektion

- Bei Verwendung von homogenen Koordinaten kann die perspektivische Projektion als lineare Abbildung mittels einer 4×4 Matrix geschrieben werden:

$$\tilde{\mathbf{P}} = \begin{pmatrix} \tilde{p}_x \\ \tilde{p}_y \\ \tilde{p}_z \end{pmatrix} = \begin{pmatrix} f \frac{p_x}{p_z} \\ f \frac{p_y}{p_z} \\ f \end{pmatrix} \in \mathbb{R}^3 \mapsto \underline{\tilde{\mathbf{P}}} = \begin{pmatrix} f p_x \\ f p_y \\ f p_z \\ p_z \end{pmatrix} \in \mathbb{H}^3$$

$$\underline{\tilde{\mathbf{P}}} = \begin{pmatrix} f p_x \\ f p_y \\ f p_z \\ p_z \end{pmatrix} = \underbrace{\begin{bmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & f & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}}_{\mathbf{A}} \begin{pmatrix} p_x \\ p_y \\ p_z \\ 1 \end{pmatrix}$$

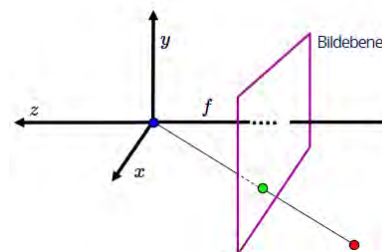
$$\underline{\tilde{\mathbf{P}}} = \mathbf{A} \underline{\mathbf{P}}$$

Perspektivische Projektion in OpenGL (s. Kapitel E)

- Kamera zeigt in die negative z-Richtung

$$\underline{\tilde{\mathbf{P}}} = \begin{pmatrix} f p_x \\ f p_y \\ f p_z \\ -p_z \end{pmatrix} = \underbrace{\begin{bmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & f & 0 \\ 0 & 0 & -1 & 0 \end{bmatrix}}_{\mathbf{A}} \begin{pmatrix} p_x \\ p_y \\ p_z \\ 1 \end{pmatrix}$$

$$\underline{\tilde{\mathbf{P}}} = \mathbf{A} \underline{\mathbf{P}}$$



Perspektivische Projektion in OpenGL (s. Kapitel E)

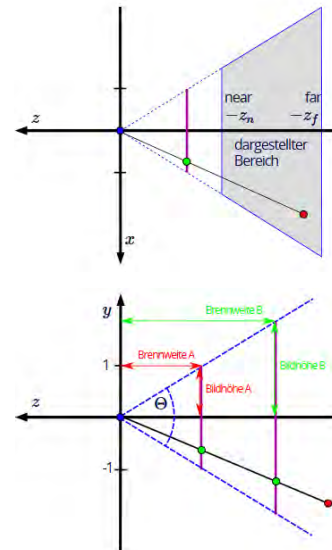
- zusätzlich gibt es eine sogenannte Near- und eine Far-Clipping-Ebene

$$\tilde{\mathbf{P}} = \underbrace{\begin{bmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & \frac{z_f + z_n}{z_n - z_f} & \frac{2z_f z_n}{z_n - z_f} \\ 0 & 0 & -1 & 0 \end{bmatrix}}_{\mathbf{A}} \begin{pmatrix} p_x \\ p_y \\ p_z \\ 1 \end{pmatrix}$$

$$\tilde{\mathbf{P}} = \mathbf{A} \mathbf{P}$$

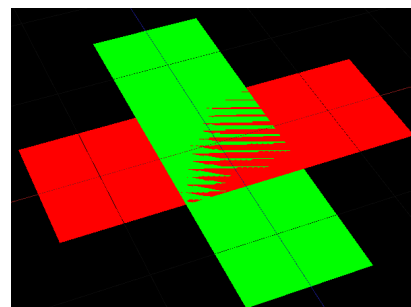
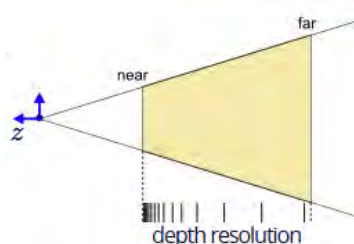
- Bildebene wird so gewählt, dass die resultierenden x- und y-Koordinaten im Bereich [-1;1] liegen

$$\frac{f}{1} = \frac{\cos(0.5 \Theta)}{\sin(0.5 \Theta)} \Leftrightarrow f = \cotan(0.5 \Theta)$$

**Bemerkung:**

- Tiefenpuffer (z-Wert) hat nur eine bestimmte Genauigkeit. Typischerweise ein Integer-Wert mit 16, 24 oder 32 Bit Genauigkeit
- Intervall [-1,0; 1,0] wird auf [0,0, 1,0] und dann auf [0, MAX_INT] abgebildet, z. B. [0, 65535] für 16 Bits
- Wert wird auf die nächstliegende Ganzzahl gerundet.

→ so genanntes „**Z-Fighting**“ durch zufällige Ungenauigkeiten bei den Z-Werten, wodurch mal das eine, mal das andere Primitiv angezeigt wird



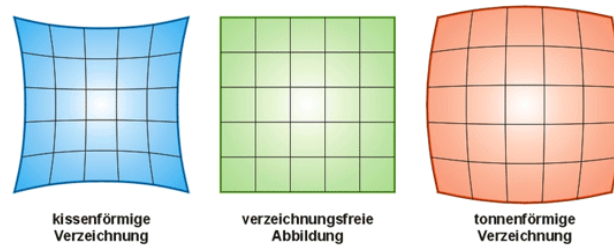
Kamerakalibrierung

Notwendigkeit:

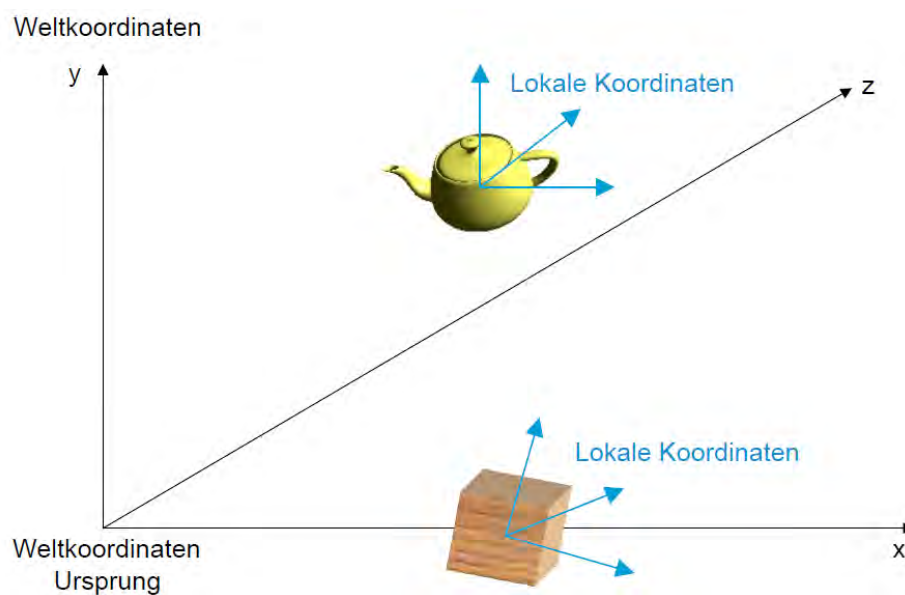
- Eine Kamera projiziert 3D-Weltpunkte auf die 2D-Bildebene
- Kompensierung von Fehlern der Kamera und der Linsen

Kalibrierung:

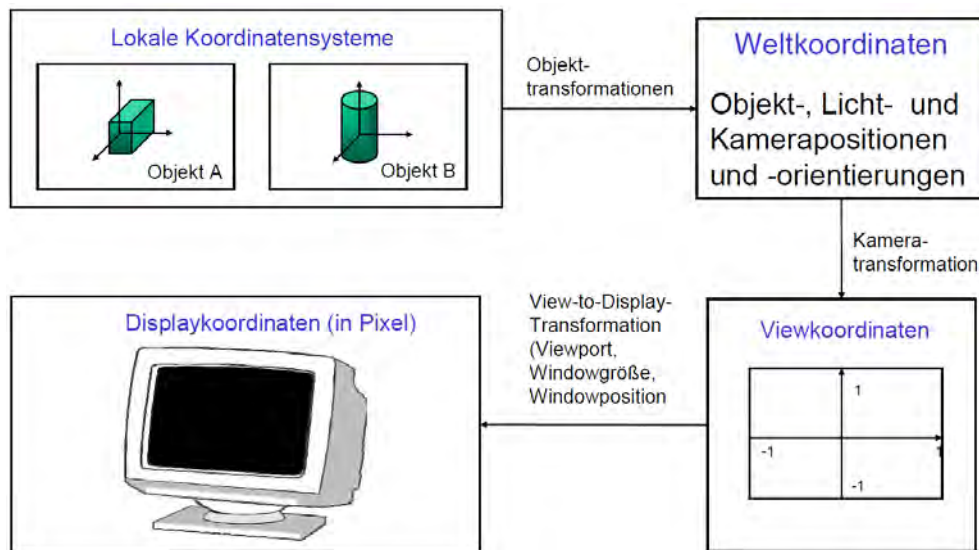
Ermittlung der Kamera-internen Größen, welche den Prozess der Projektion beeinflussen.



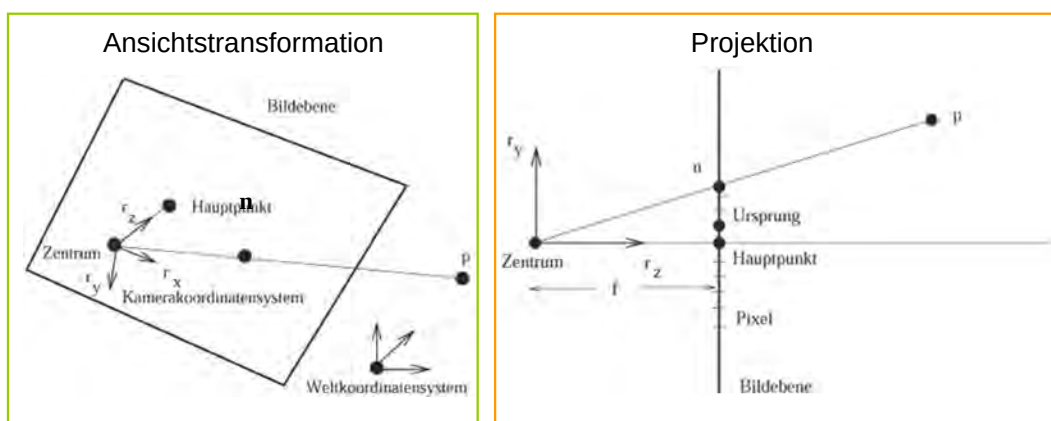
Übersicht zu den Koordinatensystemen



Übersicht zu den Koordinatensystemen



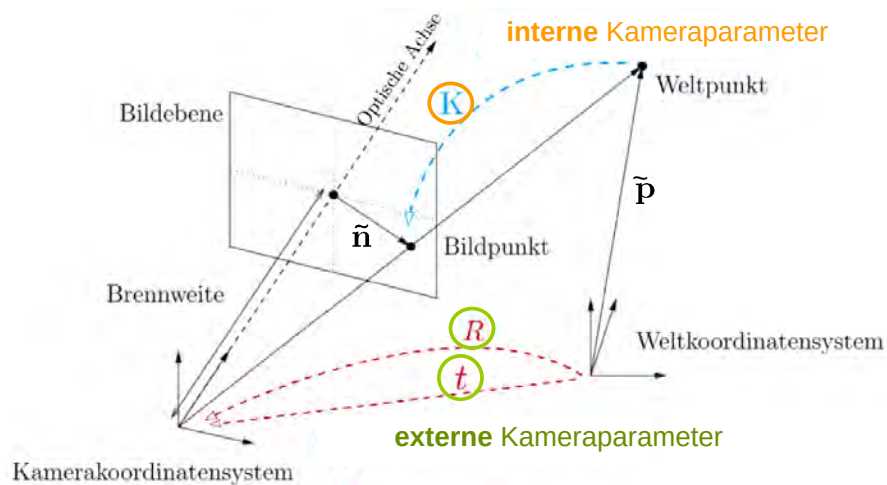
Definition der Lochkamera



Kameraparameter:

- **interne Kameraparameter** (internes Kameraabbildungsverhalten): f, u, v, d_x, d_y
- **externe Kameraparameter** (Kameraort): $\mathbf{t}, \mathbf{R}$

Kamerakoordinatensysteme

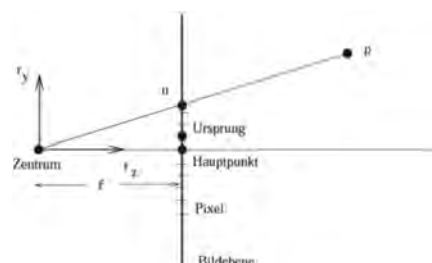


Ideale Abbildung eines Objektpunkts auf die Bildebene einer Lochkamera ohne Verzerrung

Abbildungsfunktion: $\tilde{\mathbf{n}} = \mathbf{K} \circ \mathbf{L} \tilde{\mathbf{p}}$

$\swarrow$ $\searrow$
 interne Komponente externe Komponente

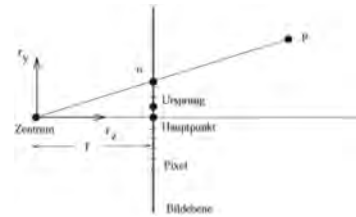
- $\tilde{\mathbf{p}}$ der zu projizierende Raumpunkt in **homogenen Weltkoordinaten** $(\mathbf{p}, 1)$
- $\tilde{\mathbf{n}} = (\tilde{n}_x, \tilde{n}_y, \tilde{n}_z)^*$ die Pixelkoordinaten der Projektion von $\mathbf{p}$ in **homogener Darstellung**
- $\mathbf{n} = (n_x, n_y)^*$ die **Pixelkoordinaten der Projektion von $\mathbf{p}$** , wobei
 $n_x = \tilde{n}_x / \tilde{n}_z$, $n_y = \tilde{n}_y / \tilde{n}_z$

**Bemerkung:**

* bezeichnet die Transponierung eines Vektors oder einer Matrix.

Abbildungsfunktion: $\tilde{n} = \mathbf{K} \circ \mathbf{L}\tilde{p}$

$\swarrow$ interne Komponente $\searrow$ externe Komponente



Interne Kamerageometrie:

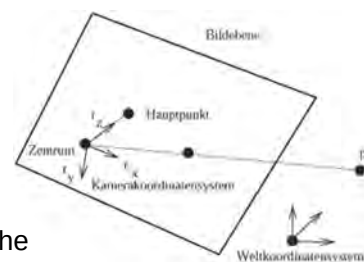
- $\mathbf{K} = \mathbf{D} \circ \mathbf{P}$ Kameraabbildung

- $\mathbf{P} = \begin{pmatrix} f & 0 & 0 \\ 0 & f & 0 \\ 0 & 0 & 1 \end{pmatrix}$ f die **effektive Brennweite** oder **Kamerakonstante**,

- $\mathbf{D} = \begin{pmatrix} 1/d_x & 0 & c_x \\ 0 & 1/d_y & c_y \\ 0 & 0 & 1 \end{pmatrix}$ d_x, d_y die Ausdehnung der Sensorelemente,
 c_x, c_y die Koordinaten des Bildmittelpunktes (Hauptpunktes) bezüglich des Bildkoordinatensystems (Pixelkoordinatensystem, dessen *Ursprung* leicht verschoben sein kann)

Abbildungsfunktion: $\tilde{n} = \mathbf{K} \circ \mathbf{L}\tilde{p}$

$\swarrow$ interne Komponente $\searrow$ externe Komponente

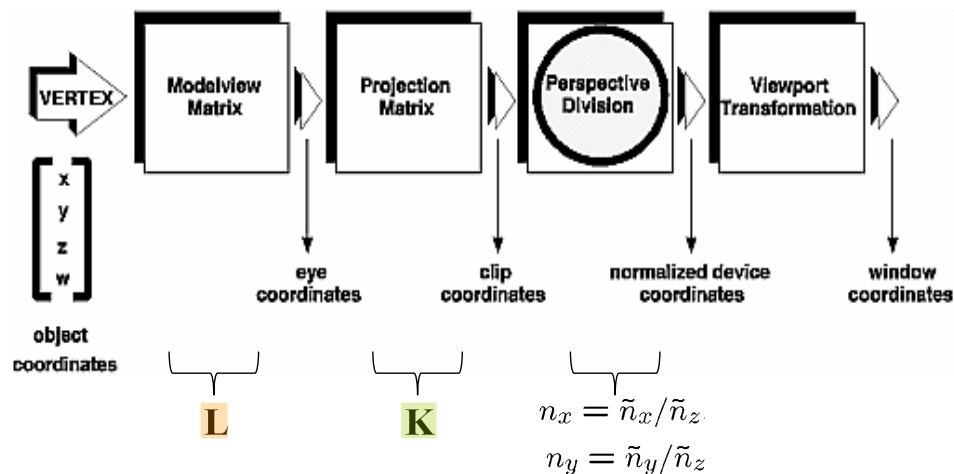


Externe Kamerageometrie:

- $\mathbf{L} = \mathbf{R}^*(\mathbf{I}_3 | -\mathbf{t})$ die Matrix des Kameraortes, welche die Transformation von Weltkoordinaten in Kamerakoordinaten repräsentiert, wobei
- $\mathbf{R} = (\mathbf{r}_x, \mathbf{r}_y, \mathbf{r}_z)$ Rotationsmatrix mit $\mathbf{r}_x, \mathbf{r}_y$ und $\mathbf{r}_z$ die orthogonalen normierten Achsenvektoren des *Kamerakoordinatensystems* in Weltkoordinaten, wobei $\mathbf{r}_z$ in Richtung der *optischen Achse* zeigt.
- $\mathbf{t}$ Translationsvektor zur Koordinatentransformation zwischen Welt- und Kamerakoordinatensystem, wobei $\mathbf{t}$ das *Projektionszentrum* in Weltkoordinaten ist,

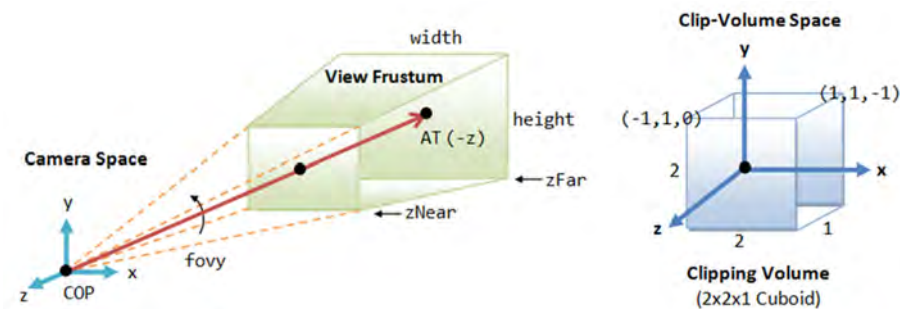
- $\mathbf{I}_3 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$

Terminologie der Bilderzeugungs-Pipeline der Computergraphik:

Abbildungsfunktion: $\tilde{n} = K \circ L \tilde{p}$ 

(Volume-)Clipping:

- Beschränkung der Szene auf einen Ausschnitt, der vom Kamerabild erfasst wird.
- Realisierung durch Clip-Ebenen, die über die Bildgrenzen in der Bildebene der Kamera (left, right, top, bottom clipping plane) und ein Intervall auf der z-Achse des Kamerakoordinatensystems (near, far clipping plane) festgelegt werden.



D.1. Grundkonzepte

- D.1.1. Einführung
- D.1.2. Bilderzeugungs-Pipelines
- D.1.3. Geometrierepräsentation
- D.1.4. Geometrieeigenschaften

D.2. Transformation

- D.2.1. Einleitung
- D.2.2. Affine Abbildungen
- D.2.3. Homogene Darstellung

D.3. Beleuchtungsberechnung

- D.3.1. Einleitung
- D.3.2. Beleuchtungsmodell von Phong
- D.3.3. Shading-Modelle

D.4. Clipping und Projektion

- D.4.1. Einleitung
- D.4.2. Clipping
- D.4.3. Picking
- D.4.4. Projektion
- D.4.5. Kamerakalibrierung

D.5. Sichtbarkeit

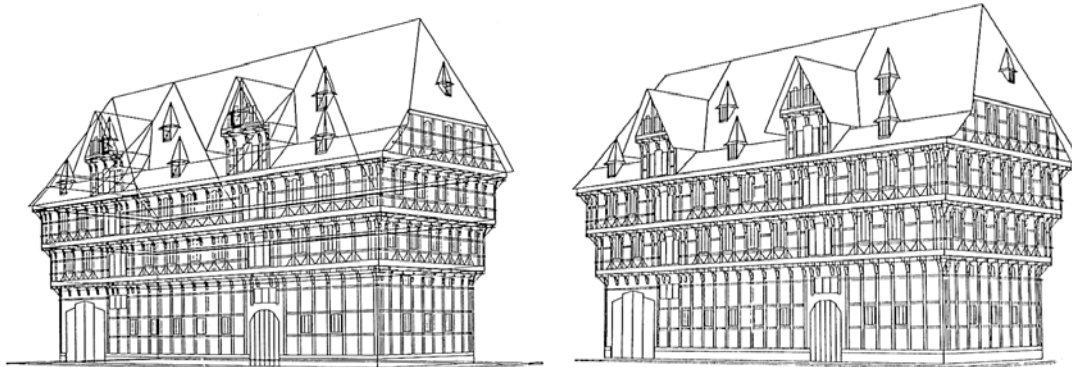
- D.5.1. Einleitung
- D.5.2. Szenenbasiertes Lösungsverfahren
 - D.5.2.1. Painter's Algorithm
 - D.5.2.2. Binäre Raumzerlegung
- D.5.3. Rasterbildbasierte Lösungsverfahren
 - D.5.3.1. Tiefenpufferverfahren
 - D.5.3.2. Scan-Line-Verfahren
 - D.5.3.3. Bereichsunterteilungsverfahren

D.6. Verrasterung

- D.6.1. Strecken in expliziter Darstellung
- D.6.2. Strecken in impliziter Darstellung
- D.6.3. Anti-Alias-Behandlung
- D.6.4. Dicke Strecken
- D.6.5. Verrasterung von Polygonen
- D.6.6. Beleuchtungsberechnung und Verrasterung
- D.6.7. Textur
- D.6.8. Flächenfüllen

Sichtbarkeitsberechnung

Beispiel:



Original

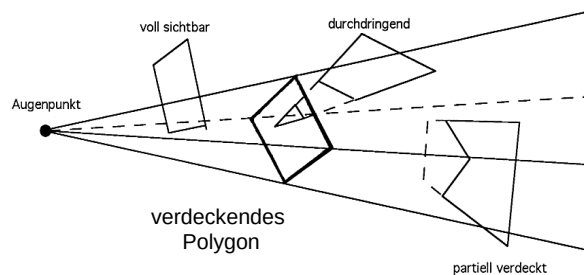
ohne verdeckte
Linien

Sichtbarkeitsberechnung für Polygone:

Gegeben: eine Szene ebener Polygone $P_1, \dots, P_m$ im $\mathbb{R}^3$, ein Augenpunkt $\mathbf{a}$.

Gesucht:

- alle von $\mathbf{a}$ aus sichtbaren Teile der Polygone
(Berechnung der sichtbaren Flächen / hidden surface elimination)
- beziehungsweise alle von $\mathbf{a}$ aus sichtbaren Kanten der Polygone
(Berechnung der sichtbaren Linien / hidden line elimination).

**Lösungsansätze:**

- **szenenbasierte Sichtbarkeitsberechnung:**

Die Sichtbarkeitsberechnung geschieht unabhängig von der Art der Bilddarstellung (Rasterbild, Liniengraphik) basiert auf der für die Szene verwendeten Geometrirepräsentation.

Vorteil: Unabhängigkeit von der Art der Bilddarstellung

Beispiele für szenenbasiertes Lösungsverfahren: Painter's Algorithm, BSP

- **bildbasierte Sichtbarkeitsberechnung:**

Die Sichtbarkeitsberechnung geschieht abhängig von der Art der Bilddarstellung.

Vorteil: Beschränkung auf die Bedürfnisse der eingesetzten Art der Bilddarstellung.

Beispiele für rasterbildbasierte Lösungsverfahren:

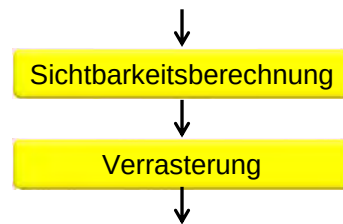
Tiefenpufferverfahren, Scan-Line-Verfahren, Bereichsunterteilungsverfahren

**Szenenbasierte Lösungsverfahren:**

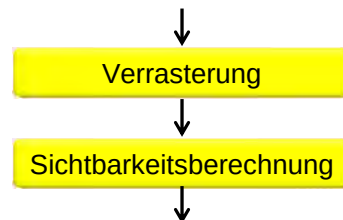
- Painter's Algorithm
 - Binäre Raumzerlegung (Binary Space Partitioning)
- } Prioritätslistenverfahren

Allgemeine Vorgehensweise:

Sortieren der Polygone entsprechend ihres Abstands vom Beobachter und anschließendes Abarbeiten in dieser Reihenfolge.

**Rasterbildbasierte Lösungsverfahren:**

- Tiefenpufferverfahren (z-Buffer)
- Scan-Line-Verfahren
- Bereichsunterteilungsverfahren

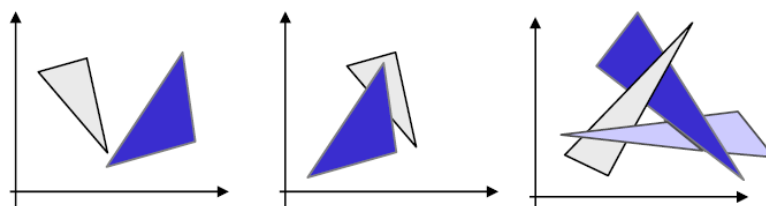
**Painter's Algorithm**

Die Szenenpolygone werden nach fallender Entfernung ihres am geringsten vom Beobachter entfernten Eckpunktes auf die Bildebene projiziert, verrastert und dann gezeichnet.

Dadurch überdecken im allgemeinen näherliegende Polygone die weiter entfernt liegenden.

Bemerkung:

Der Painter's Algorithmus liefert im Allgemeinen keine korrekte Lösung, da etwa in den gezeigten Beispielen eine Sortierung nicht möglich ist:

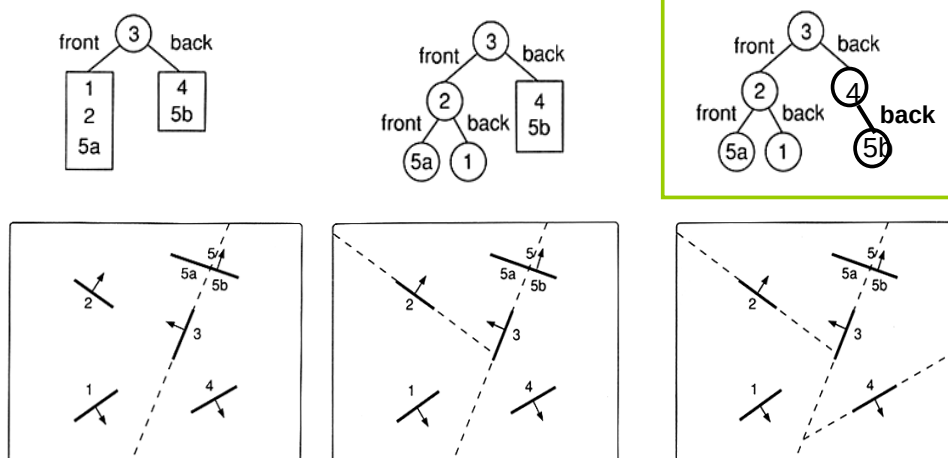


Binäre Raumzerlegung - BSP = Binary Space Partition**Vorverarbeitungsphase:**

Aufbau einer Raumzerlegung in Form eines Suchbaums, des BSP-Baums, in einem Vorverarbeitungsschritt. Der BSP-Baum erlaubt es dann, für jede Projektion schnell, d.h. in linearer Zeit, eine Anordnung der Polygone so zu finden, dass sie, in dieser Reihenfolge projiziert, das Verdeckungsproblem lösen.

Aufbau des BSP-Baums:

Nimm ein Polygon der Szene und teile die Szene durch seine Ebene in die beiden Teilszenen, die aus den Polygonen in den beiden durch die Ebene induzierten Teilräumen liegen. Polygone, die die Ebene schneiden, werden durch die Ebene in Teilpolygone zerlegt. Das ausgewählte Polygon bildet die Wurzel des Baums. Sie besitzt zwei Teilbäume, deren Wurzeln entsprechend aus den beiden Teilszenen bestimmt werden.

Beispiel (2D-Darstellung):



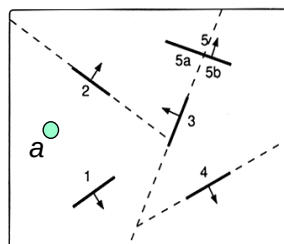
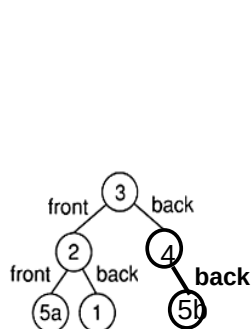
Sichtbarkeitsberechnungsphase:

Durchlaufe den BSP-Baum so, dass zunächst der Teilbaum bzgl. eines Knotens durchlaufen wird, der die Teilszene enthält, in deren Halbraum sich der Augenpunkt der Projektion nicht befindet. Bevor der zweite Teilbaum durchlaufen wird, wird das Polygon des Knotens vorne an die auszugebende Prioritätsliste angefügt.

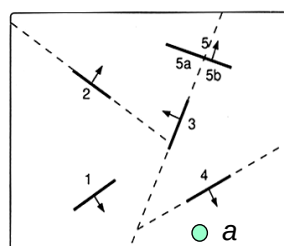
Die resultierende Prioritätsliste enthält die Polygone sortiert nach fallender Entfernung vom Augenpunkt, d.h. ergibt, in dieser Reihenfolge gezeichnet, die korrekte Verdeckung.



Beispiel: Sichtbarkeitsberechnung mittels des BSP-Baums



Ergebnis für den Augenpunkt a:
4, 5b, 3, 5a, 2, 1



Ergebnis für den Augenpunkt a:
5a, 2, 1, 3, 5b, 4

**Bemerkung:**

1. Der in der Vorverarbeitungsphase berechnete BSP-Baum beschleunigt die Sichtbarkeitsberechnung. Der Zeitaufwand der Sichtbarkeitsberechnung ist linear in der Anzahl der Knoten des BSP-Baums. Der Aufwand für das Sortieren der Polygone entfällt bei der Sichtbarkeitsberechnung.
2. Das Verfahren der binären Raumzerlegung eignet sich besonders dafür, eine statische Szene aus verschiedenen Blickrichtungen zu inspizieren.

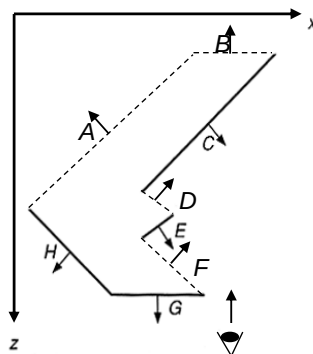
QUELLE

Foley, van Dam, Feiner, Hughes, Kap. 15.5.2

**Back-face culling:**

Die Polygone, die keinen Normalenvektoranteil entgegen der Blickrichtung haben (back-faces, gestrichelt) (A,B,D,F) werden entfernt, die anderen Polygone (front-facing polygons) (C,E,G,H) werden weiter behandelt.

Die reduzierte Anzahl von Polygonen erlaubt eine schnellere Sichtbarkeitsberechnung.

Beispiel:

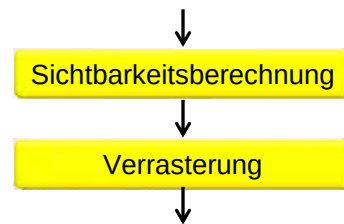
Entscheidung über das Vorzeichen des Skalarprodukts zwischen Blickrichtungsvektor und Polygonnormalenvektor

Szenenbasierte Lösungsverfahren:

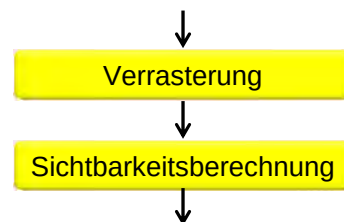
- Painter's Algorithm
 - Binäre Raumzerlegung (Binary Space Partitioning)
- } Prioritätslistenverfahren

Allgemeine Vorgehensweise:

Sortieren der Polygone entsprechend ihres Abstands vom Beobachter und anschließendes Abarbeiten in dieser Reihenfolge.

**Rasterbildbasierte Lösungsverfahren:**

- Tiefenpufferverfahren (z-Buffer)
- Scan-Line-Verfahren
- Bereichsunterteilungsverfahren

**Vorgehensweise beim Tiefenpufferverfahren:**

- Projektion und Verrasterung der Polygone der Szene nacheinander auf der Bildebene.
- Für jeden Bildpunkt eines Polygons Bestimmung seiner Entfernung zum Augenpunkt (Tiefe) und Vergleich der Tiefe mit einem für den Bildpunkt bereits gespeicherten Tiefenwert.
- Bei geringerer Tiefe Zuweisung der Tiefe an diesen Bildpunkt und Registrierung dieses Polygon als an diesem Bildpunkt sichtbar.
- Ablegen der Sichtbarkeitsinformation bzw. die Tiefeninformation in einem Feld (Array) mit einer der Bildauflösung entsprechenden Größe.

Bildpuffer



Tiefenpuffer



Visualisierung der Tiefe durch Grauwerte: dunkel = nahe, hell = weit entfernt.

QUELLE: Wikipedia.



Tiefenpuffer/depth buffer/z-buffer-Algorithmus

Eingabe: Eine 3D-Szene aus Flächen, eine Projektion, eine Bildauflösung $n \times m$.

Ausgabe: für jedes Pixel des $n \times m$ Rasterbildes die dort sichtbare Fläche.

Ablauf: Datenstrukturen:

VAR Tiefenpuffer: **ARRAY** $[1..n, 1..m]$ **OF** numbertype;

VAR Bild: **ARRAY** $[1..n, 1..m]$ **OF** colortype;

BEGIN

(* initialisiere den Tiefenpuffer *)

FOR $i := 1$ **TO** m **DO**

FOR $j := 1$ **TO** n **DO BEGIN**

 Tiefenpuffer $[i, j] := \infty$;

 Bild $[i, j] :=$ "Hintergrundfarbe"

END;



Verrasterung mit dem
Scan-Line-Verfahren

(* berechne das Bild *)

FOR alle Flächen f der Szene **DO**

FOR alle Pixel $[i, j]$ in der Projektion von f **DO BEGIN**

 bestimme die Tiefe w of f am Pixel $[i, j]$; } Sichtbarkeitsberechnung

IF $w <$ Tiefenpuffer $[i, j]$ **THEN BEGIN**

 Tiefenpuffer $[i, j] := w$;

 (* registriere die neue sichtbare Fläche *)

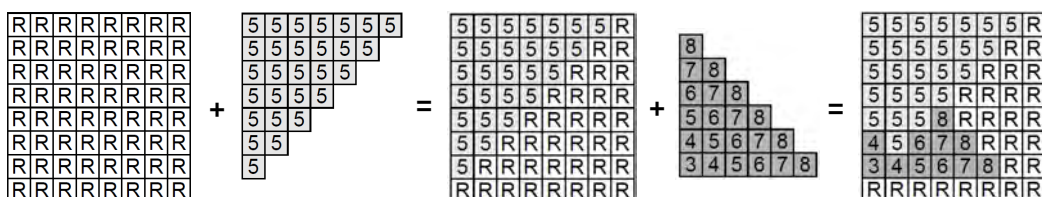
 Bild $[i, j] := f$

END;

(* jetzt enthält "Bild" die sichtbaren Flächen *)

END.

Beispiel:



(* berechne das Bild *)

FOR alle Flächen **f** der Szene **DO**

FOR alle Pixel $[i, j]$ in der Projektion von **f** **DO BEGIN**

bestimme die Tiefe w of **f** am Pixel $[i, j]$; } Sichtbarkeitsberechnung

IF $w < \text{Tiefenpuffer}[i, j]$ **THEN BEGIN**

Tiefenpuffer $[i, j] := w$;

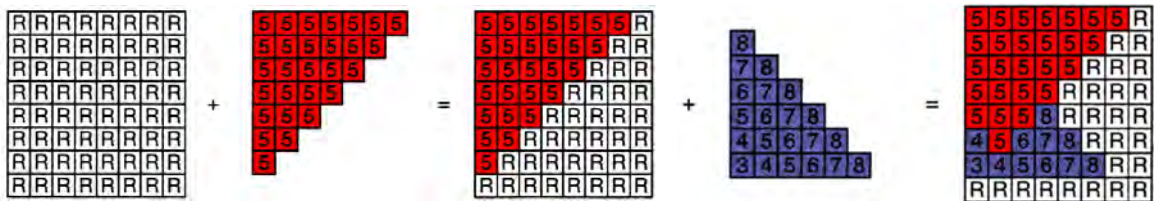
(* registriere die neue sichtbare Fläche *)

Bild $[i, j] := f$

END;

(* jetzt enthält "Bild" die sichtbaren Flächen *)

END.

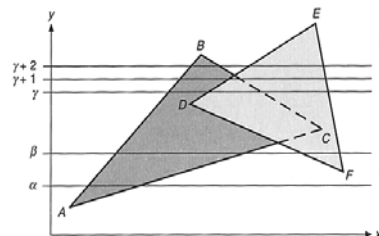
Beispiel:**Algorithmus zum Scan-Line-Verfahren**

Eingabe: Eine 3D-Szene aus Flächen, eine Projektion, eine Bildauflösung $n \times m$.

Ausgabe: für jedes Pixel des $n \times m$ Rasterbildes die dort sichtbare Fläche.

Ablauf: Arbeite das Bild zeilenweise ab und führe für jede Zeile aus:

- Bestimme für die aktuelle Zeile die von ihr geschnittenen Polygonkanten;
- Arbeite die Polygonkanten nach aufsteigender x-Koordinate ihres Schnittpunkts mit der Scan-Line ab:
 - Bestimme das am Schnittpunkt der aktuellen Polygonkante sichtbar werdende Polygon (dies kann auch das bisher sichtbar gewesene sein);
 - Zeichne die Bildpixel der Scan-Line zwischen letztem und aktuellem Schnittpunkt mit der Farbe des am letzten Schnittpunkt sichtbar gewordenen Polygons.



Bemerkungen:

1. Es wird eine Datenstruktur der aktuell aktiven Polygone längs der Scan-Line gehalten.
2. Bestimmung des Ein-/Austritts eines Polygons zur Aktualisierung der Datenstruktur der aktiven Polygone durch alternierendes Setzen eines In/Out-Flags.

Alternativ kann folgende Beobachtung verwendet werden:

Falls die Polygonkanten so orientiert sind, dass das Polygoninnere links davon angeordnet ist, liegt eine Eintrittskante vor, wenn der y-Wert des Anfangspunkts größer als der des Endpunkts ist, ansonsten eine Austrittskante.

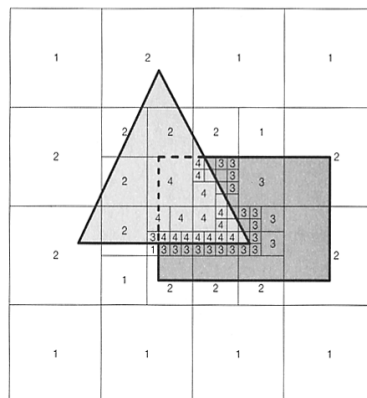
3. Die Zuordnung eines ganzen Abschnitts zwischen zwei Kanten auf der Scan-Line ist nur korrekt, wenn sich die Polygone nicht wechselseitig durchdringen. Andernfalls müssen die in diesem Abschnitt aktiven Polygone auf Durchdringung getestet werden. Dies kann durch Vergleich der Tiefen an den Abschnittsenden geschehen.

vgl. Fellner, Kap. 14.5 / Foley, van Dam, Feiner, Hughes, Kap. 15.6

Bereichsunterteilungsverfahren (von Warnock)

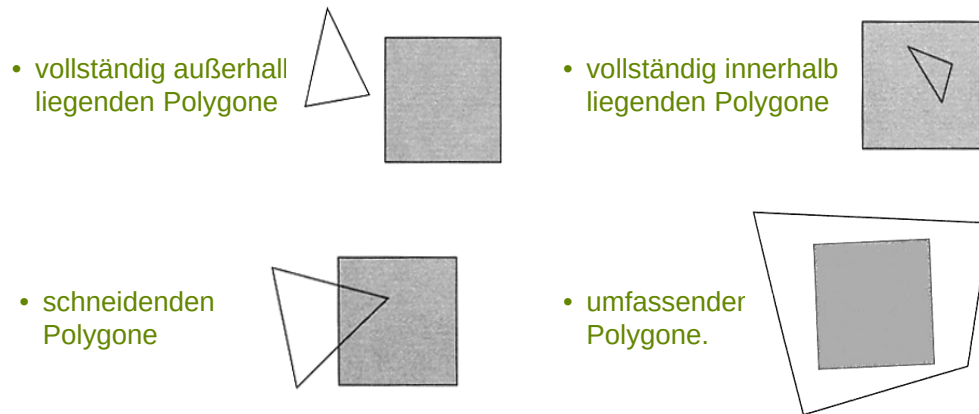
Ein- und Ausgabe wie zuvor.

Vorgehensweise: Aufteilen der Bildebene in Teilbilder, die so einfach sind, dass die Sichtbarkeitsberechnung unmittelbar ausgeführt werden kann.



Ablauf:

Zerlege das Bild sukzessive durch Vierteilung und berechne für jedes Teilbild die Mengen der:

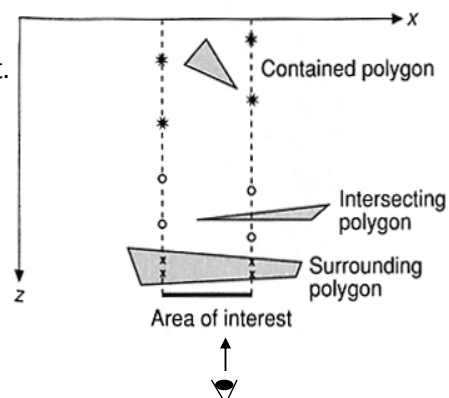


Ein Teilbild wird nicht weiter unterteilt, wenn eine der folgenden Bedingungen erfüllt ist:

- Alle Polygone liegen außerhalb.
- Liste enthält nur ein Polygon, das vollständig innerhalb, schneidend oder umfassend ist.
- Umfassendes Polygon verdeckt innerhalb des Teilbilds alle anderen Polygone.
- Das Teilbild hat Pixelgröße.

→ In diesen Fällen wird das Teilbild gezeichnet.

Ist keine dieser Bedingungen erfüllt, dann werden für das Teilbild die vollständig außerhalb liegenden Polygone und die Polygone, die im Teilbild von einem umfassenden Polygon verdeckt werden, entfernt. Danach wird das Teilbild zerlegt und mit den neuen Teilbildern entsprechend verfahren.



D.1. Grundkonzepte

- D.1.1. Einführung
- D.1.2. Bilderzeugungs-Pipelines
- D.1.3. Geometrieprepräsentation
- D.1.4. Geometrieeigenschaften

D.2. Transformation

- D.2.1. Einleitung
- D.2.2. Affine Abbildungen
- D.2.3. Homogene Darstellung

D.3. Beleuchtungsberechnung

- D.3.1. Einleitung
- D.3.2. Beleuchtungsmodell von Phong
- D.3.3. Shading-Modelle

D.4. Clipping und Projektion

- D.4.1. Einleitung
- D.4.2. Clipping
- D.4.3. Picking
- D.4.4. Projektion
- D.4.5. Kamerakalibrierung

D.5. Sichtbarkeit

- D.5.1. Einleitung
- D.5.2. Szenenbasiertes Lösungsverfahren
 - D.5.2.1. Painter's Algorithm
 - D.5.2.2. Binäre Raumzerlegung
- D.5.3. Rasterbildbasierte Lösungsverfahren
 - D.5.3.1. Tiefenpufferverfahren
 - D.5.3.2. Scan-Line-Verfahren
 - D.5.3.3. Bereichsunterteilungsverfahren

D.6. Verrasterung

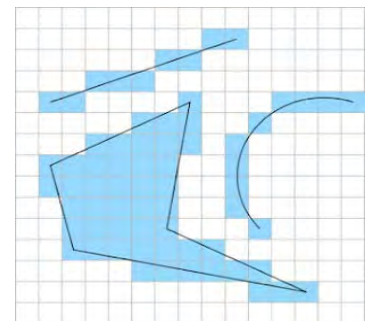
- D.6.1. Strecken in expliziter Darstellung
- D.6.2. Strecken in impliziter Darstellung
- D.6.3. Anti-Alias-Behandlung
- D.6.4. Dicke Strecken
- D.6.5. Verrasterung von Polygonen
- D.6.6. Beleuchtungsberechnung und Verrasterung
- D.6.7. Textur
- D.6.8. Flächenfüllen

D.6. Verrasterung

 Einleitung**Konzept der Verrasterung:**

Gegeben: ein kontinuierliches geometrisches Objekt O .

Gesucht: eine Approximation von O durch Rasterpunkte, die bei hinreichend feiner Rasterauflösung wie O aussieht.

**Lösungsmöglichkeit****Zwei Schritte:**

1. Approximation von O durch Polygonzüge oder Polygone (Dreiecke)
2. Verrasterung von Strecken beziehungsweise Polygonen (Dreiecke)



Verrasterung von Strecken in expliziter Darstellung ($a \cdot x + b$)

Gegeben: eine Strecke mit Steigung zwischen -1 und 1 und mit ganzzahligen Endpunkten **p** und **q** auf dem Rastergitter.

Gesucht: Eine Folge von ganzzahligen Punkten, welche die Strecke approximieren.

Ablauf: $a := \frac{q_y - p_y}{q_x - p_x}$, $b := \frac{p_y(q_x - p_x) - (q_y - p_y)p_x}{q_x - p_x}$, $f(x) := ax + b$.

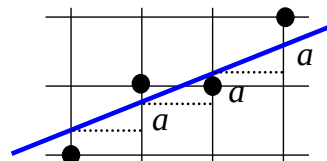
FOR $i := p_x$ **TO** q_x **DO**

BEGIN

Zeichne den Punkt (i , $\text{round}(f(i))$);

$f(i+1) := f(i) + a$;

END.



Bemerkung:

1. Die Einschränkung auf eine Steigung zwischen -1 und 1 vermeidet Rasterlücken.
2. Für Strecken mit anderer Steigung geschieht die Verrasterung entlang der anderen Koordinatenrichtung.
3. Durch die inkrementelle Rechnung werden teure Operationen wie die Multiplikation und Division vermieden.



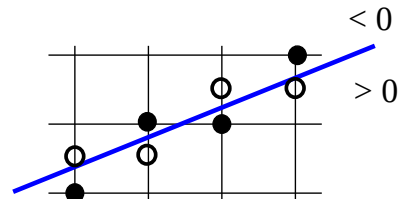
Verrasterung von Strecken in impliziter Darstellung

Algorithmus

Gegeben: eine Strecke mit Steigung zwischen 0 und 1 sowie mit ganzzahligen Endpunkten **p** und **q** auf dem Rastergitter (**p** links von **q**).

Gesucht: Eine Folge von ganzzahligen Punkten, die die Strecke approximieren.

Ablauf: $a := qy - py$, $b := -(qx - px)$, $c := -bpy - apx$,
 $F(x, y) := a \cdot x + b \cdot y + c$.
 Zeichne (px, py) ; $j := py$;
FOR $i := px + 1$ **TO** qx **DO**
BEGIN
 IF $F(i, j + 0.5) > 0$ **THEN** $j := j + 1$;
 zeichne (i, j)
END.



Bemerkung: Die Testgröße $F_d(i, j) := F(i, j + 0.5)$ kann inkrementell berechnet werden - entsprechende Realisierung des Algorithmus ist unter dem Namen „**Bresenham-Algorithmus**“ bekannt.

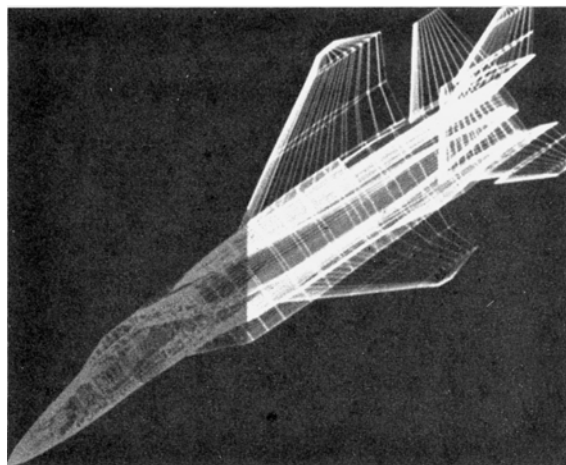


Alias-Problem: stufiges Aussehen und „Zerfallen“ von verrasterten Strecken;
 Grund: nicht korrekte Abtastung

Lösung: bei Rasterdisplays mit mehreren Intensitätsstufen Verwendung der Intensitätsstufen zur Glättung (**Anti-Aliasing**)

Beispiel:

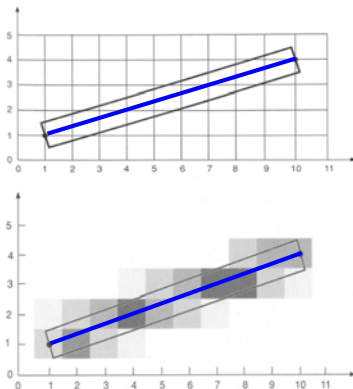
Geglättete Linienzeichnung: der vordere Bereich ist nicht geglättet, der hintere Bereich ist geglättet.



Beispiel für Anti-Aliasing

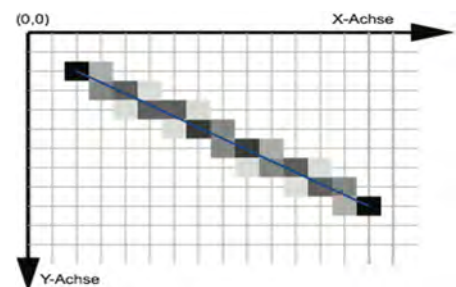
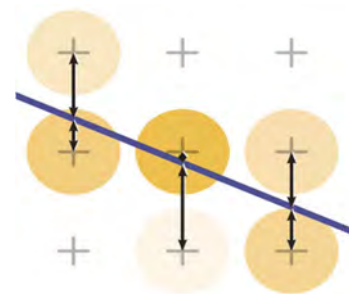
Glättung durch Verwendung von Graustufen - Helligkeit wächst mit dem Abstand des Pixels von der Strecke. In diesem Beispiel ist die Helligkeit eines Pixels proportional zur Fläche des Schnitts der zum Rechteck vergrößerten Strecke mit dem quadratischen Pixelgebiet.

Entsprechend kann bei Farbdarstellung durch Überblendung von Strecken- und Hintergrundfarbe verfahren werden.



Wu's Antialiasing-Ansatz

- Schleife über alle x-Werte
- Ermittlung von zwei Pixel, die der Ideallinie am nächsten liegen
→ normalerweise etwas oberhalb und unterhalb
- Grauwerte in Abhängigkeit der Entfernung
→ auf der Linie: 100% und 0%
→ gleicher Abstand: 50% und 50%
- Grauwerte für diese zwei Pixel festlegen



Antialiasing im Allgemeinen

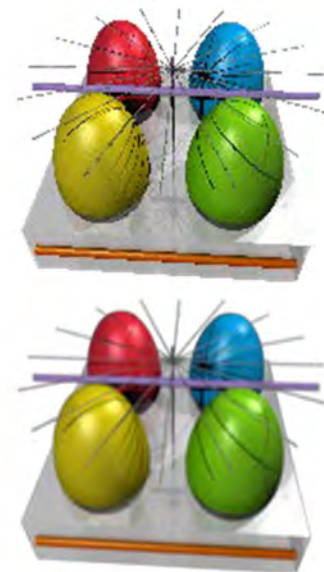
- Problem: Harte Kanten in der Computergrafik
- Entsprechen einer unendlich hohen Raumfrequenz
- Verstoß gegen das Abtasttheorem (s. Kapitel C)

Allgemeiner Ansatz: Supersampling

Idee:

- Rendering eines Bildes mit einer höheren Auflösung
- Skalieren auf die gewünschte Größe
- Interpolieren von Pixelwerten

Direkt unterstützt vom OpenGL (s. Kapitel E)

**Algorithmus** zur Pixelwiederholung (**Dicke Strecken**)

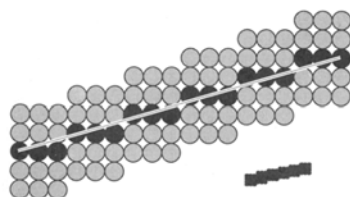
Gegeben: Eine Strecke, eine Linienbreite

Gesucht: Eine gerasterte Darstellung der Strecke in der gewünschten Linienbreite.

Ablauf:

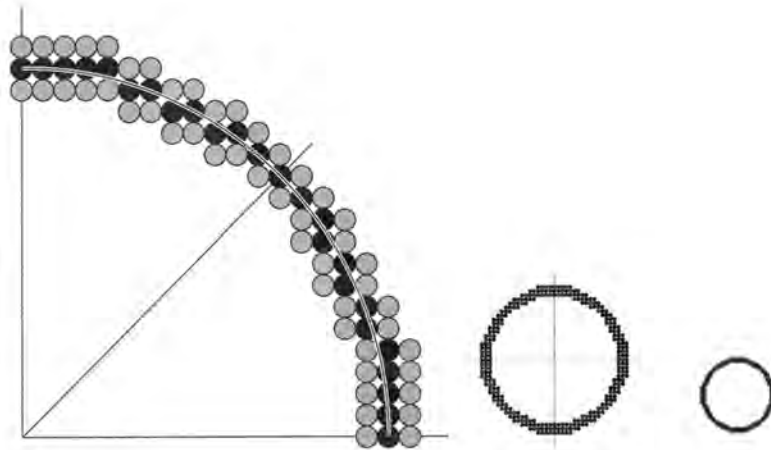
- Zeichne die "dünne" Strecke nach einem üblichen Scan-Konvertierungsverfahren;
- Falls die Steigung der Kurve zwischen -1 und 1 liegt, dupliziere die Pixel der dünnen Strecke symmetrisch in den Spalten;
- Andernfalls dupliziere die Pixel der dünnen Strecke symmetrisch auf den Zeilen.

Beispiel:



Nachteile:

- Die Strecke endet stets entweder waagrecht oder senkrecht. Dadurch können "dünne" Stellen entstehen.

Beispiel:**Nachteile:**

- Die Liniendicke hängt von der Steigung ab: Falls d die Dicke einer horizontalen oder vertikalen Strecke ist, hat eine Strecke mit Steigung 1 die Dicke $d/\sqrt{2}$.
- Durch die symmetrische Anordnung der duplizierten Pixel um die dünne Strecke können nur Strecken "ungerader Dicke" gezeichnet werden.

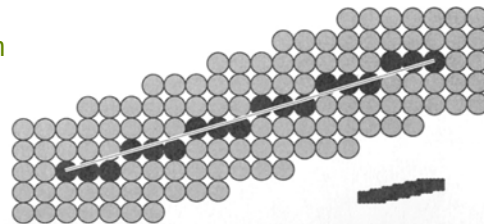
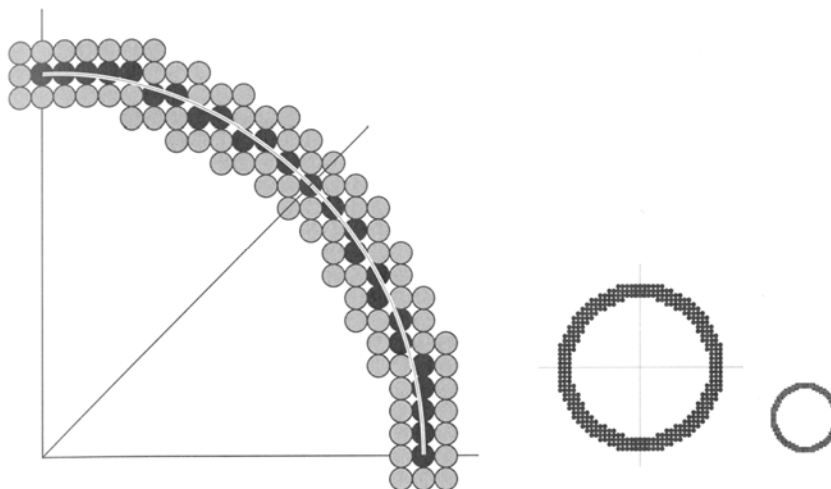
Vorteil:

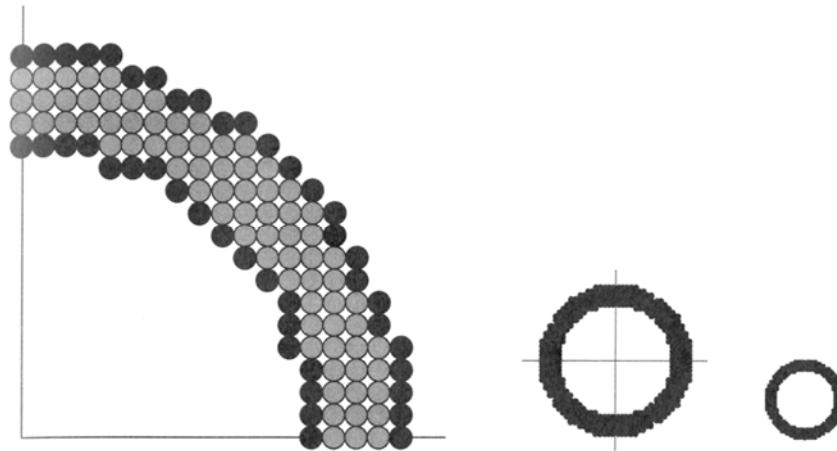
einfaches Verfahren, das für nicht zu dicke Strecken akzeptabel ist.

Algorithmus zum Pinselstrichverfahren**Gegeben:** Eine Strecke, eine Linienbreite**Gesucht:** Eine gerasterte Darstellung der Strecke in der gewünschten Linienbreite.**Ablauf:**

Bewege eine gerasterte Pinselform entlang der gegebenen Strecke, wobei die Pixel der mit einem Scan-Konvertierungsverfahren für dünne Strecken verrasterten Kurve im Zentrum des Pinselmusters liegen.

Die dabei überdeckten Pixel definieren die dicke Strecke.

Beispiel: rechteckige Pixelform**Beispiel: rechteckige Pixelform**

Beispiel: kreisförmige Pinselform**Vorteil der kreisförmigen Pinselform:**

- (1) Anders als bei der rechteckigen Pinselform hängt die Breite der dicken Kurve nicht von der Steigung ab.
- (2) Effiziente Realisierung des Pinselstrichverfahrens durch Vermeiden des mehrfachen Zeichnens von Pixeln:
- (3) zeichne nur den Teil des Randes der Pinselform, der außerhalb der bisher bezeichneten Pixel liegt:
 - bei rechteckiger Pinselform und nordöstlicher Zeichenrichtung genügt das Zeichnen des nördlichen bzw. östlichen Randes
 - bei kreisförmiger Pinselform genügt das Zeichnen eines Halbkreises, der auf der Gerade senkrecht zur zu zeichnenden Strecke durch das aktuelle Pixel beginnt.

DETAILS:

W.D. Fellner, Computergraphik, B.I. Wissenschaftsverlag, S. 135

Verrasterung von Polygonen

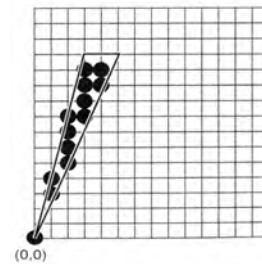
Gegeben: ein Polygon P , ein Punktraster, das P vollständig umfasst.
Die Polygoneckpunkte werden auf dem Raster angenommen.

Gesucht: die Punkte des Rasters, die in P oder auf einer linken oder unteren Kante liegen.

Schwachstelle dieser Definition: Problem bei Spitzen (Alias-Problem).

Verfahren zur Polygonverrasterung:

- Scan-Line-Verfahren
(Foley et al., Kap. 3.6, Fellner, Kap. 6.4)
- randbezogenes Füllen
(Fellner, Kap. 6.4)



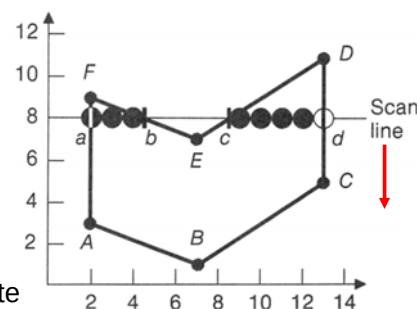
Algorithmus zur Polygonverrasterung durch das Scan-Line-Verfahren

Datenstrukturen:

- (1) X-Datenstruktur:** enthält die von der aktuell aktiven Zeile geschnittenen Kanten, die sogenannten aktiven Kanten.

Operationen:

- **Insert:** Einfügen einer Kante in die Datenstruktur
- **Delete:** Entfernen einer angegebenen Kante aus der Datenstruktur.



- (2) Y-Datenstruktur:** enthält die noch nicht bearbeiteten Polygonkanten absteigend sortiert nach ihrem größten y-Wert.

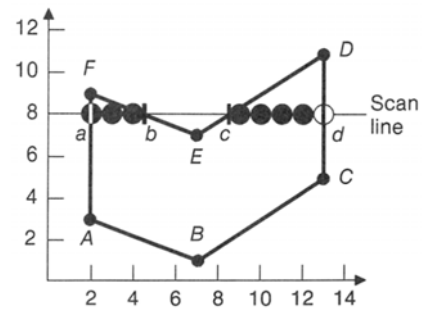
Operationen:

- **Initialize:** sortiertes Einfügen aller Polygonkanten.
- **MinDelete:** liefert das erste Element und entfernt es aus der Datenstruktur.

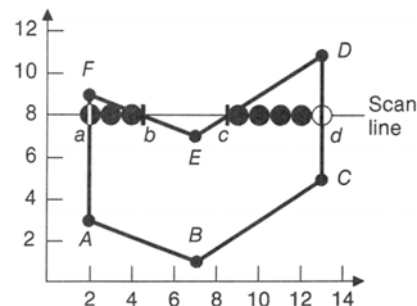
Ablauf:

Arbeite das Bild zeilenweise ab und führe für jede Zeile aus:

- Bestimme für die aktuelle Zeile die von ihr geschnittenen Polygonkanten:
 - Entferne die Kanten aus der X -Datenstruktur, welche die aktuelle Zeile nicht mehr schneiden.
 - Füge die Kanten aus der Y -Datenstruktur in die X -Datenstruktur ein, die die aktuelle Zeile jetzt schneiden, und entferne sie aus der y -Datenstruktur.



- Bestimme die Intervalle auf der Zeile, die durch die Schnittpunkte induziert werden:
 - Halte die Kanten sortiert in der X -Datenstruktur. Das bedeutet, dass beim Einfügen an die entsprechend der Sortierung richtige Stelle eingefügt werden muss.
 - Die die Intervalle definierenden Pixel können für schon vorhandene Kante inkrementell aus denen der vorigen Zeile berechnet werden.
- Gib die Rasterpunkte in den Intervallen aus, die im Inneren des Polygons liegen:
 - Die im Innern liegenden Intervalle ergeben sich aus der sortierten Reihenfolge der Pixel alternierend zu den äußeren Intervallen.



**Bemerkungen:**

1. Repräsentation einer Kante:

Objekt aus

- den Koordinaten des oberen Endpunkts (der mit maximaler y-Koordinate),
- der Differenz der y-Koordinaten des oberen und des unteren Endpunktes,
- dem x-Inkrement für die inkrementelle Berechnung ihrer Verrasterung.

2. Horizontale Kanten werden ignoriert.

3. Bei Polygoneckpunkten werden die Kanten berücksichtigt, die dort einen y-minimalen Endpunkt haben, die mit y-maximalem Endpunkt nicht.

**Beleuchtungsberechnung und Verrasterung****Aufgabe:**

Berücksichtigung der Beleuchtungsberechnung bei der Verrasterung

Eingabe:

(1) Ein Dreieck mit

- optischen Attributen entsprechend zum Phong-Beleuchtungsmodell an jedem Eckpunkt
- einem Normalenvektor an jedem Eckpunkt

(2) Punktlichtquellen

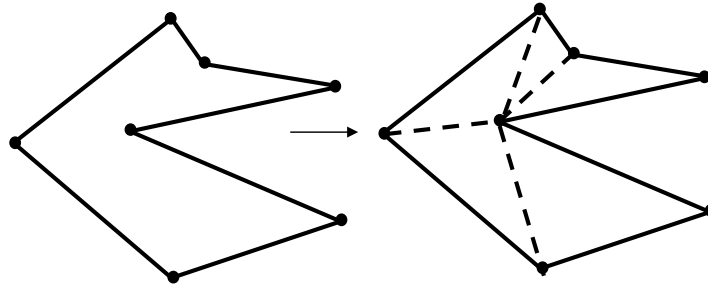
(3) ein Augenpunkt und eine Bildebene

Ausgabe: Ein Beleuchtungswert an jedem Pixel des projizierten Dreiecks

**Bemerkung:**

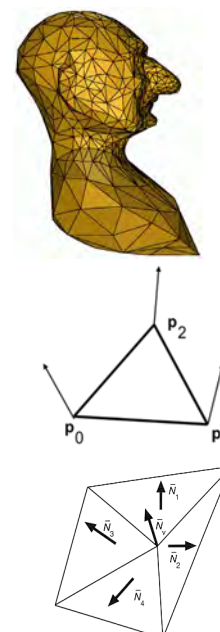
Im Fall von beliebigen Polygonen anstelle von Dreiecken können diese durch Polygontriangulierung in eine Menge von sich nicht scheidenden Dreiecken überführt werden.

Bsp.: Triangulierung eines Polygons durch Einfügen von Sehnen

**Bemerkung:**

Definition von Normalenvektoren an den Polygoneckpunkten:

- Bei einem einzelnen Dreieck: Die Normalenvektoren der Eckpunkte stimmen mit dem Normalenvektor des Dreiecks überein.
- Bei einem Dreieck in einem Dreiecksnetz:
 - Die Normalenvektoren können den Normalenvektoren der gekrümmten Fläche entsprechen, die durch das Netz repräsentiert wird.
 - Die „gekrümmten“ Normalenvektoren können als gewichtetes Mittel der Normalenvektoren der inzidenten Dreiecke der Eckpunkte approximiert werden.

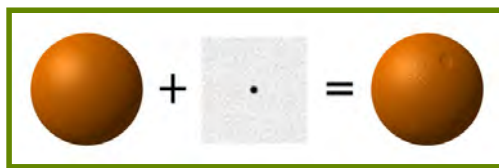


Textur in der Computergraphik:

Feinstruktur, die durch Funktionen beschrieben wird, welche die Abhängigkeit der Parameter des Beleuchtungsmodells vom Ort beschreiben.

Beispiele:

- Beeinflussung der Oberflächenfarbe über die Parameter der diffusen Reflexion
- Beeinflussung der Transparenz über den Transparenzparameter, z.B. für Nebeneffekte
- Beeinflussung des Normalenvektors der Oberfläche, um Oberflächenrauheiten zu modellieren („Bump-Mapping“)



Beeinflussung der Oberflächenfarbe



Beeinflussung des Normalenvektors

Textur in der Computergraphik:

Feinstruktur, die durch Funktionen beschrieben wird, welche die Abhängigkeit der Parameter des Beleuchtungsmodells vom Ort beschreiben.

Beispiele:

- Beeinflussung der Oberflächenfarbe über die Parameter der diffusen Reflexion
- Beeinflussung der Transparenz über den Transparenzparameter, z.B. für Nebeneffekte
- Beeinflussung des Normalenvektors der Oberfläche, um Oberflächenrauheiten zu modellieren („Bump-Mapping“)

Texturarten:

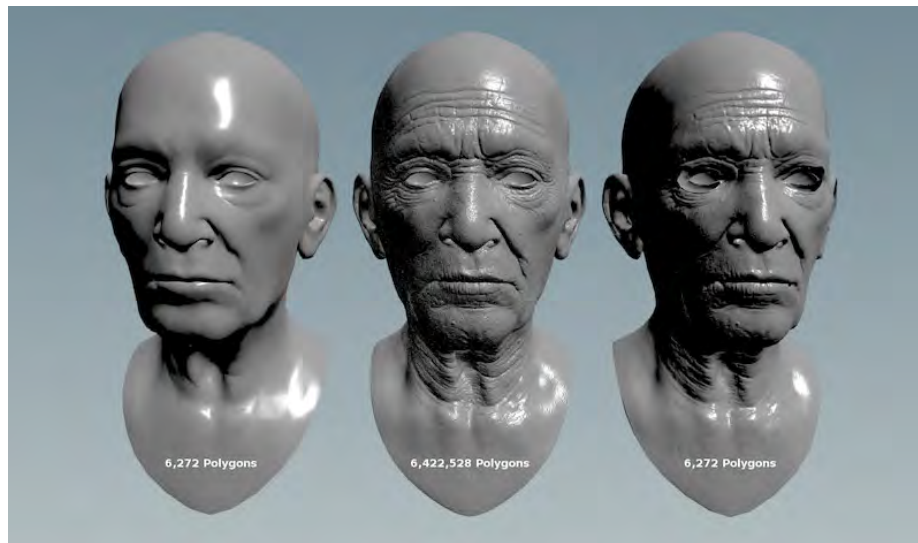
- **Oberflächentextur**
Voraussetzung: Oberflächenkoordinatensystem
- **Volumentextur**
Voraussetzung: Raumkoordinatensystem



Beeinflussung der Oberflächenfarbe



Beeinflussung des Normalenvektors

**Texturverfahren:**

- **simulierte Textur**

Funktion, die als Parameter eine Koordinatenangabe im Texturkoordinatensystem hat. Für den durch die Koordinatenangabe gegebenen Punkt wird als Ergebnis der entsprechende Texturwert zurückgegeben.

Vorteil: keine Schwierigkeit mit Alias-Effekten

Nachteil: Simulierte Texturen wirken häufig wenig natürlich.

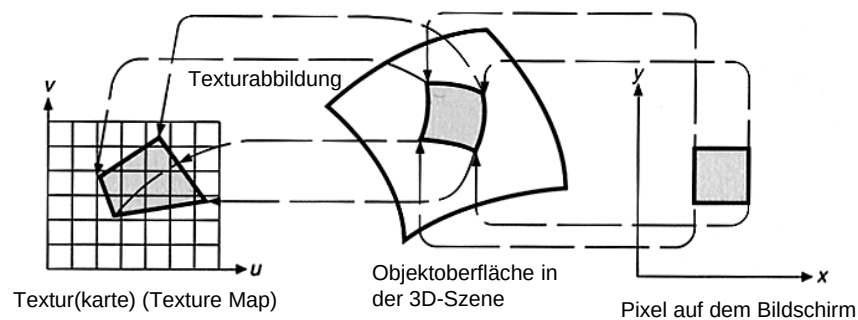
LITERATUR:

D.J. Heeger, J.R. Bergen, Pyramid-based Texture Analysis/Synthesis, Proc. SIGGRAPH 1995, 229-238

Li-Yi Wei, Sylvain Lefebvre, Vivek Kwatra, Greg Turk, State of the Art in Example-based Texture Synthesis, Eurographics 2009, State of the Art Report, EG-STAR – 2009

<http://www-sop.inria.fr/reves/Basilic/2009/WLKT09/>

- **abgebildete Textur (Texture Mapping)**



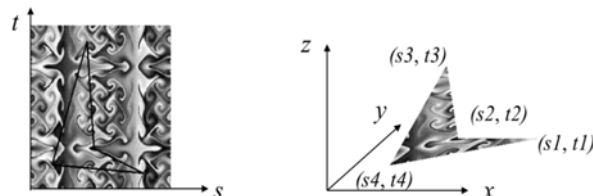
Die Textur wird als Rastermatrix (Texturkarte) vorgegeben, die z.B. durch Digitalisieren natürlicher Texturen gewonnen wurde.

Vorteil: natürliches Aussehen

Nachteil: Anti-Alias-Maßnahmen sind erforderlich; Bsp.: Mip-Mapping

Texturabbildung bei Polygonen:

- In der Szenenbeschreibung: Zuweisung von Texturkoordinaten an jeden Eckpunkt



- Bei der Bildberechnung: Bestimmung einer Texturcoordinate an jedem Pixel durch Interpolation der Texturkoordinaten an den Eckpunkten durch Anwendung der Basismethode der linearen Interpolation bei der Verrasterung.
- An jedem Pixel Verwendung der Texturwerts bei der Beleuchtungsberechnung, der sich an der entsprechenden Stelle in der Texturkarte befindet.

Problem: Generierung von Texturen beliebiger Größe aus einem gegebenen Bild

Lösung: Wiederholungstextur

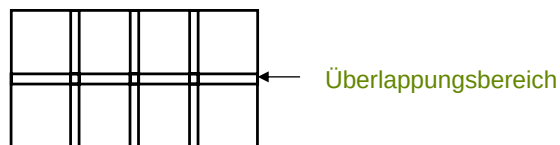
Wiederholungstextur:

entsteht durch periodische Wiederholung des gegebenen Texturbildes („Kachel“)

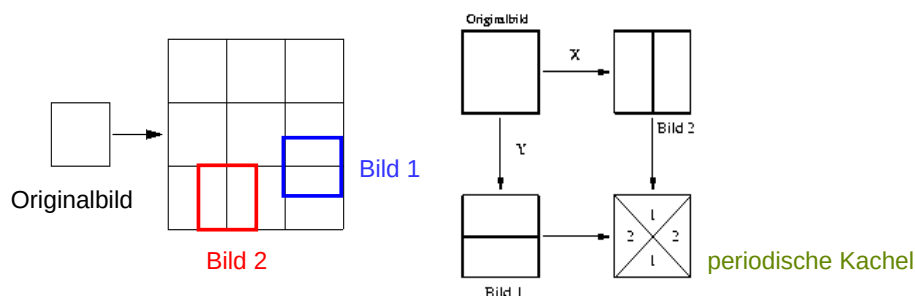
Problem: Elimination von Kanten und des Periodenartefakts

Lösungsmöglichkeit:

1. Überlappen der Kacheln und Überblenden der Überlappungsbereiche



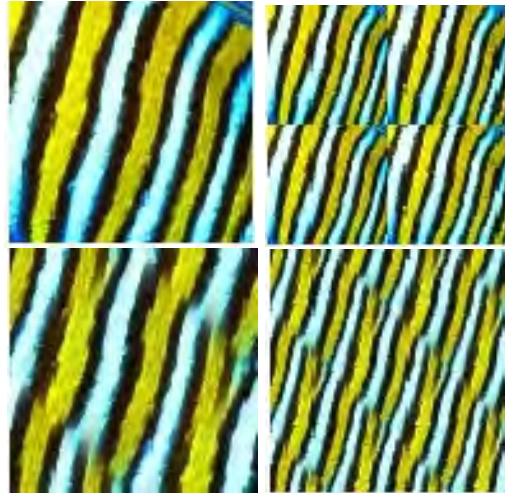
2. Schaffung einer periodischen Kachel mit geglätteten Kanten im Inneren



Vorteile:

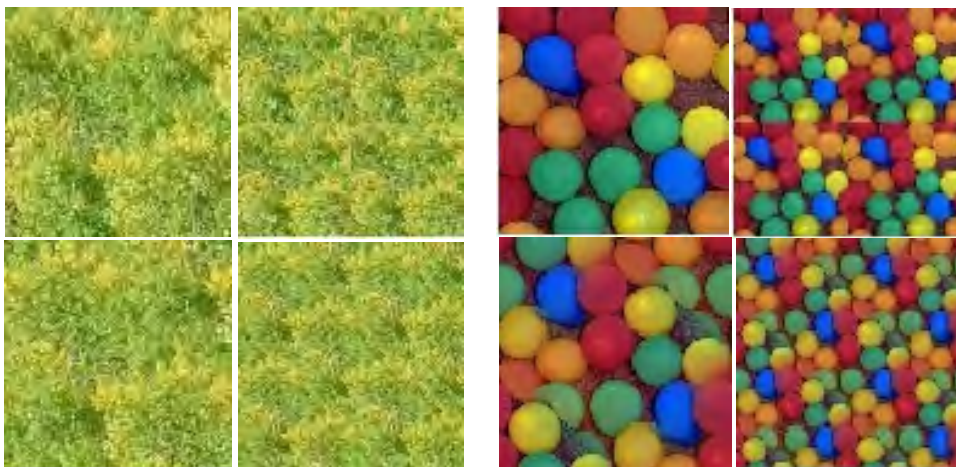
- Verlagerung der unstetigen Kanten und der Glättung ins Innere, wodurch eine Vorverarbeitung der Kachel möglich wird
- schräge Lage der unstetigen Kanten, für die das Auge weniger empfindlich zu sein scheint

Bsp.:



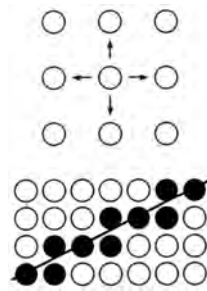
Original	direktes Aneinandersetzen des Originals
„Periodisierung“	Aneinandersetzen der Periodisierung

Bsp.:

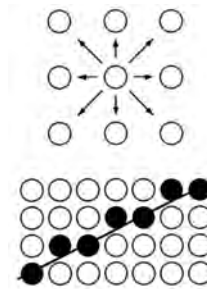


4-zusammenhängende Pixelkurve:

eine Folge von Pixeln, wobei das nächste Pixel durch waagrechtes oder senkrechtes Fortschreiten erreicht wird.

**8-zusammenhängende Pixelkurve:**

eine Folge von Pixeln, wobei das nächste Pixel durch waagrechtes, senkrechtes oder diagonales Fortschreiten erreicht wird.

**Algorithmus zum Füllen eines Flächengebiets** (Foley, Kap. 19.5.1)**Eingabe:**

eine 8-zusammenhängende geschlossene Pixelkurve, ein Startpunkt (x_0, y_0) im Inneren der Kurve. Die Pixelwerte RW auf dem Rand seien alle gleich. NW sei ein bisher nicht verwendeter neuer Wert.

Ausgabe:

das Rasterbild, bei dem die inneren Pixel den Wert NW der Randkurve haben.



```
Ablauf: Unteralgorithmus Randverrastern( $x, y$ , Randwert, Neuwert);  
BEGIN  
   $W := \text{Pixel}(x, y);$   
  IF  $W \neq \text{Randwert}$  AND  $W \neq \text{Neuwert}$  THEN  
    BEGIN  
      gib Pixel( $x, y$ ) den Wert Neuwert;  
      Randverrastern( $x, y - 1$ , Randwert, Neuwert);  
      Randverrastern( $x, y + 1$ , Randwert, Neuwert);  
      Randverrastern( $x - 1, y$ , Randwert, Neuwert);  
      Randverrastern( $x + 1, y$ , Randwert, Neuwert);  
    END  
  END.  
Hauptalgorithmus:  
BEGIN  
  Randverrastern( $x_0, y_0$ , RW, NW)  
END.
```

**Bemerkung:**

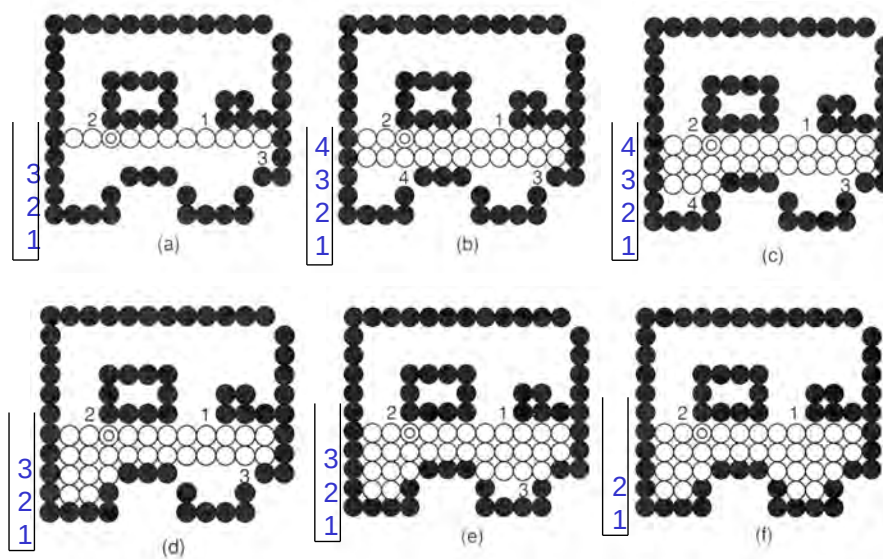
Der Algorithmus für das innenbezogene und randbezogene Füllen ist durch die vielfache Rekursion häufig ineffizient wegen des Speicherbedarfs für den Stack.

bessere Lösung: *Span-Fill-Algorithmus*

Arbeitsweise des Span-Fill-Algorithmus:

- Füllen ganzer Zeilenintervalle (Spans) und Übernahme von Pixeln, welche die Intervallgrenzen auf den benachbarten Zeilen liefern in einen Keller.
- Das als nächstes zu füllende Intervall wird vom Keller geholt.

Beispiel: Span-Fill-Algorithmus



DETAILS: Foley, Kap. 19.5.2

Vergleich zwischen den 2D- und 3D-Bilderzeugungs-Pipelines

